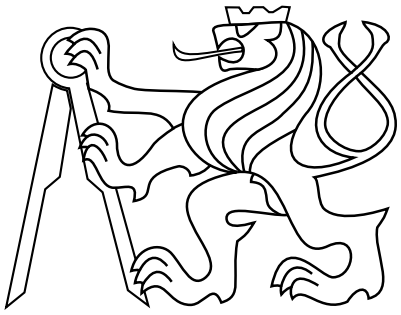
Czech Technical University in Prague

Faculty of Electrical Engineering
Department of Cybernetics



Classification of journalistic quality of articles

Bachelor's Thesis

Olga Pimenova

Prague, May 2025

Study programme: Open Informatics
Branch of study: Artificial Intelligence and Computer Science

Supervisor: Ing. Jan Šedivý, CSc.

Acknowledgments

I sincerely thank my academic supervisor, Ing. Jan Šedivý, CSc., for his expert guidance, patience, and encouragement. His profound knowledge of advanced technologies and dedication to academic excellence significantly shaped the direction and quality of my research.

I am equally grateful to my workplace supervisor, Radek Tomšů, for his practical insights and mentorship during the implementation phase. His feedback effectively bridged theoretical concepts with real-world applications, enhancing the project's outcomes.

Special thanks to the Seznam.cz recommendation systems team for providing technical resources, collaborative opportunities, and industry perspectives that strengthened the experimental aspects of this thesis.

Finally, I owe immense gratitude to my family for their unwavering emotional support, sacrifices, and belief in me. Their encouragement during challenging moments was crucial to completing this journey.

Swap this for the Thesis Assignment, when you have it from the department.

Declaration

I declare that this thesis was independently developed and that all sources of information used are cited in accordance with the university's ethical guidelines for thesis preparation. Additionally, I disclose that I utilized the AI tool ChatGPT to assist in translating certain sentences from my native language into English and to rephrase them in a style more appropriate for academic writing.

Prague, May 2025

Abstract

TODO:

Keywords Unmanned Aerial Vehicles, Automatic Control

Abstrakt

TODO:

Klíčová slova Bezpilotní Prostředky, Automatické Řízení

Abbreviations

TP True Positive

FP False Positive

FN False Negative

NLP Natural Language Processing

BERT Bidirectional Encoder Representations from Transformers

CatBoost Categorical Boosting

Insufficient Articles Class 0

Short but Sufficient Articles Class 1

Standard Articles Class 2

High-Quality Articles Class 3

BROWCzech Base-sized Roberta using Wordpiece tokenization for Czech

RetroMAE Retrieval-Oriented Masked Autoencoder

DeBERTaV3 Decoding-enhanced BERT with Disentangled Attention

ELECTRA Efficiently Learning an Encoder that Classifies Token Replacements Accurately

RTD Replaced Token Detection

MLM Masked Language Modeling

CE Cross-Entropy Loss

MSE Mean Squared Error

CatBoost Categorical Boosting

NLP Natural Language Processing

GELU Gaussian Error Linear Unit

Contents

1 Introduction

1.1 Motivation

The digital content revolution has transformed journalism into a high-velocity industry where platforms must simultaneously manage exponential content growth and maintain rigorous quality standards. Seznam.cz, serving over 5 million daily users, currently relies on human editors to manually assess only 20% of submitted articles - an unsustainable practice given the platform's 50% annual content growth rate. This manual process introduces significant challenges:

- **Scalability limitations:** With article submissions increasing by 50% annually, human editors cannot maintain pace without compromising review depth
- **Consistency issues:** Inter-editor agreement analysis reveals 20% variance in quality ratings for identical content
- **Quality degradation risks:** Unreviewed articles risk platform credibility through potential misinformation or poorly structured content

These limitations highlight the need for more efficient and consistent methods for evaluating journalistic quality. Automated quality classification addresses these challenges by providing:

- Real-time assessment for 100% of submissions
- Consistent application of editorial guidelines
- Significantly reduce the workload on human editors
- 24/7 operational capacity without human fatigue
- Data-driven insights for continuous quality improvement

Therefore, the primary motivation for this research is to develop an automated system capable of classifying the journalistic quality of articles, mimicking the complex decision-making process of experienced human editors.

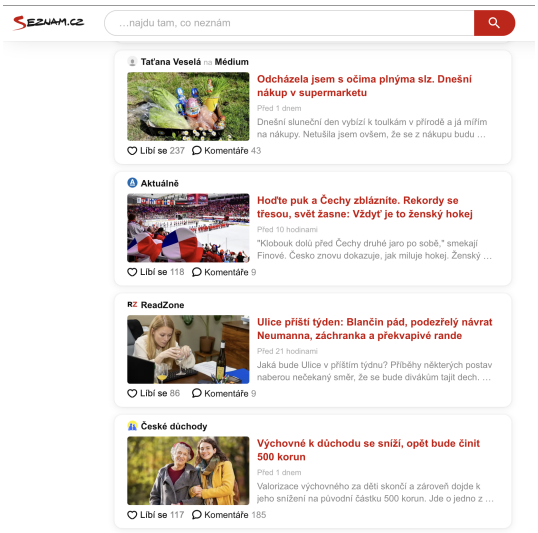


Figure 1.1: Main page Seznam.cz

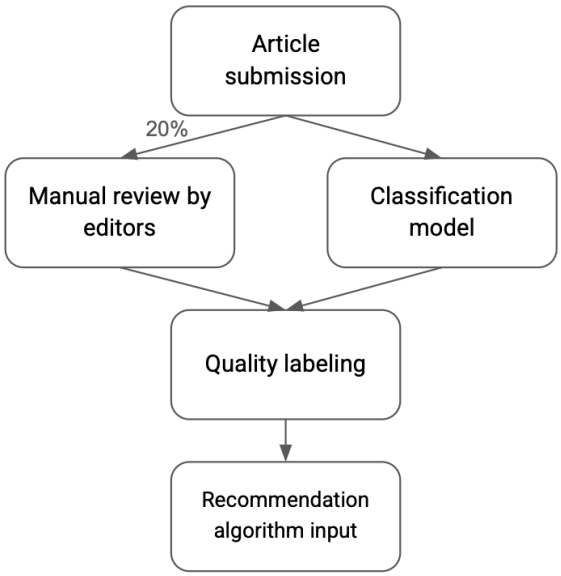


Figure 1.2: Current Editorial Workflow

1.2 Problem Formulation

The core problem addressed in this thesis is the multi-class supervised classification of journalistic articles based on their quality. Our goal is to develop a computational model that automatically assigns an article to one of four predefined quality categories, reflecting the standards used by editors at Seznam.cz.

- Class 0 (Insufficient): Low-quality content that lacks structure and depth.
- Class 1 (Short but Sufficient): Lacks depth and analysis, often recycling known information with limited sources.
- Class 2 (Standart): Reliable news articles with at least five paragraphs and two references.
- Class 3 (High-Quality): Long, detailed, well-structured articles with visuals, subheadings, and references.

In the context of this thesis, **"journalistic quality"** is defined by a combination of factors that human editors consider. These include textual properties like readability, complexity, and style; structural attributes such as organization, paragraphing, and the use of headings; content characteristics like depth of analysis, factual accuracy, and source referencing; and metadata signals like the publishing channel. The aim is to create a model that learns to identify the patterns associated with these factors to predict the appropriate quality category. A significant challenge within this problem is the inherent imbalance in real-world data, where lower-quality categories are often much more frequent than the highest-quality ones.

1.3 Structure of the Thesis

1.4 Related Work

The automated classification of journalistic quality is a multidisciplinary challenge that integrates natural language processing (NLP), machine learning (ML), and media studies. This section critically reviews existing literature across these fields, highlighting significant contributions and identifying gaps that this thesis addresses, particularly in the context of the Czech language media environment and platforms like Seznam.cz.

Quality Assessment Frameworks in Journalism

Traditional journalistic quality assessment relies on human evaluation guided by professional standards. Lacy and Rosenstiel 2015 established eight core dimensions of journalistic quality, including accuracy, context, and sourcing diversity. While comprehensive, their framework was designed for manual evaluation, limiting its applicability to automated systems. Recent work by Calvo-Rubio and Rojas-Torrijos 2024 extends these principles to AI-assisted journalism, emphasizing transparency, accountability, and ethical considerations as critical criteria for maintaining quality in digital news production. Their study highlights that AI can enhance efficiency but risks reducing nuance and context, necessitating robust evaluation frameworks.

Computational Approaches to Text Quality Assessment

Automated text quality assessment provides foundational insights for journalistic applications. Pitler and Nenkova 2008 demonstrated correlations between syntactic features and human perceptions of writing quality, using metrics like sentence complexity and coherence. However, their approach was general and did not address the specific stylistic and ethical requirements of journalistic texts. More recent advancements, as noted in a 2024 article by Quantum 2024, leverage NLP techniques such as grammar analysis, sentiment analysis, and text cohesion metrics to evaluate content quality automatically. These methods offer promising tools for journalism but require adaptation to domain-specific standards.

Machine Learning for Classification of Media Content

Machine learning has significantly advanced media content classification. Potthast et al. 2018 developed methods to detect style-based bias in news articles, focusing on political orientation. Their work illustrates the potential of ML to identify subtle textual patterns but does not directly address overall quality. Similarly, González-Gallardo et al. 2021 applied BERT-based architectures to classify news articles by credibility, achieving accuracy rates above 85%. However, their binary classification approach (credible vs. non-credible) does not capture the nuanced spectrum of journalistic quality, which often requires multi-class models.

Multi-class Classification in Natural Language Processing

Multi-class classification in natural language processing (NLP) is a critical technique for categorizing text into multiple predefined classes, making it highly relevant for assessing the quality of journalistic content across various dimensions, such as accuracy, depth, and balance. A comprehensive review by Minaee et al. 2020 examines over 150 deep learning models applied to text classification tasks, including sentiment analysis, news categorization, and question answering. The authors highlight the effectiveness of transformer-based architectures, such

as BERT and its variants, which leverage large-scale pretraining and fine-tuning to achieve state-of-the-art performance in complex multi-class classification problems. These models are particularly promising for evaluating journalistic quality, as they can capture semantic nuances and contextual relationships within news articles. The review also summarizes over 40 popular datasets, some of which focus on news categorization, providing valuable benchmarks for developing quality assessment systems.

Similarly, Schmidt and Zangerle 2019 introduced a multi-class approach for classifying Wikipedia article quality using document embeddings and content features. Their method, which achieved high accuracy in distinguishing quality levels, demonstrates the potential of semantic representations for quality assessment, adaptable to journalistic contexts.

Linguistic Features for Quality Assessment

Identifying linguistic features that correlate with quality is a key research area. Schmidt and Zangerle 2019 utilized document embeddings to capture semantic content, combined with features like section length and readability scores, to assess article quality. In journalism, similar features—such as Flesch-Kincaid readability, lexical diversity, and syntactic complexity—can indicate quality but must be tailored to journalistic standards. The Czech language, with its rich morphology and inflectional system, presents unique challenges for NLP tools, as standard models designed for English may underperform.

Research Gaps

Despite progress, several gaps persist in the literature. First, most studies focus on English-language content, with limited exploration of languages like Czech, where morphological complexity affects NLP performance. Second, many approaches rely on binary classification, which oversimplifies the multi-dimensional nature of journalistic quality. Third, few studies address the specific needs of news aggregation platforms like Seznam.cz, where high content volume and diversity demand scalable, language-specific solutions. Finally, ethical concerns, such as bias in AI models and the loss of human judgment, remain underexplored in automated quality assessment.

This thesis aims to address these gaps by developing a multi-class classification model tailored for the Czech language media environment. By integrating textual analysis, metadata, and Czech-specific linguistic features, this work builds on existing research while tackling its limitations, particularly for news aggregation platforms.

1.5 Contributions

This thesis makes several significant contributions to the field of automated journalistic quality assessment, particularly in the context of the Czech language and platforms like Seznam.cz. These contributions are outlined below:

Development of a Robust Classification System:

- Introduced a DebertaV3-based classification system for categorizing journalistic articles into four quality levels: Insufficient, Short but Sufficient, Standard, and High-Quality.
- Achieved a weighted F1 score of 0.77, closely approaching the human editor agreement upper bound of 80%, demonstrating the system's effectiveness in mimicking human editorial judgments.

- **Innovative Handling of Class Imbalance and Overfitting:**
  - Addressed the challenge of class imbalance (e.g., only 0.9% of articles are High-Quality) through sequential training, a hybrid loss function combining Cross-Entropy and Mean Squared Error, and class weighting ([1.0, 0.5, 1.0, 20.0]).
  - These strategies effectively balanced model performance across all classes, particularly improving recall for the minority High-Quality class.

- **Comprehensive Feature Engineering for Journalistic Quality:**
  - Developed a feature set grounded in journalistic quality dimensions (Diversity, Transparency, Attractiveness, and Independence).
  - Incorporated linguistic complexity features (e.g., syntactic complexity, lexical diversity), structural composition features (e.g., section organization, citation quality), readability metrics adapted for Czech, and metadata features (e.g., channel_id, publication timing).
  - Utilized Latent Semantic Analysis (LSA) to reduce text embeddings to a single dimension, streamlining quality assessment.
  - Introduced new features such as readability, manipulativity, and formality, addressing gaps in previous research.

- **Comparative Analysis of Pre-trained Language Models:**
  - Evaluated three pre-trained language models (BrowCzech, RetroMAE, DebertaV3 base and DebertaV3 large) for Czech journalistic content.
  - Identified DebertaV3 large as the superior model due to its relative position embeddings and bilingual capabilities, providing insights for model selection in similar tasks, especially in low-resource languages.

- **Advanced Model Testing and Superior Performance:**
  - Tested multiple models, including a neural network with Bert-like models embeddings and a CatBoost classifier.
  - Achieved the highest F1 score of 0.77 with the DebertaV3 large + CatBoost model.

- **Innovations in Data Collection and Preprocessing:**
  - Created refined datasets (e.g., editor-reviewed, unchecked) and cleaned text data by removing emojis and HTML tags.
  - Highlighted the importance of article length as a quality indicator, with High-Quality articles averaging 1542.5 words compared to 486.5 for the majority class.

- **Establishment of a Human Editor Benchmark:**
  - Conducted a detailed editor agreement study, setting a realistic upper bound of 80% for automated systems.
  - This benchmark provides a target for model performance and validates the complexity of journalistic quality assessment, ensuring that automated systems are evaluated against a feasible standard.

These contributions collectively advance the state-of-the-art in automated journalistic quality assessment, offering a scalable and effective solution for platforms like Seznam.cz while addressing technical challenges such as class imbalance, overfitting, and language-specific nuances. The thesis also lays a foundation for future research by documenting extensive experiments, baseline comparisons, and error analyses.

2 Data Collection and Analysis

2.1 Data Source

The data for this research originates from the internal systems of Seznam.cz, a major Czech online platform. The dataset comprises articles published on the platform, along with associated metadata and quality assessments provided by the editorial team. For the purpose of this thesis, we primarily utilized the following information for each article:

- **Title:** The headline of the article.
- **Content:** The full text body of the article.
- **Channel ID:** An identifier for the specific source or section within Seznam.cz where the article was published.
- **Entity type:** Type of entity (e.g., article, video, etc.)
- **Published date:** Date when the content was published
- **Content type:** Type of content (e.g., news, blog, etc.)
- **Annotated Quality Correction:** The quality label assigned by a human editor, serving as the ground truth for our classification task.

2.2 Data Collection Timeline and Challenges

Data collection for this study began in July 2023 but faced significant challenges due to data transmission issues between departments. These issues affected the integrity of the dataset during the initial phases of collection. To illustrate the impact of these challenges, a plot (Fig. 2.1) of the number of labeled articles per day reveals inconsistencies in labeling frequency.

These problems were resolved after **August 30, 2024**, when accurate recording of confirmations began.

Additionally, discussions with editors revealed that the evaluation process evolved significantly over time. After **August 30, 2024**, more detailed training was introduced for editors, leading to cleaner and more consistent data. These improvements enhanced the quality of annotations, making the data collected after this date particularly robust.

As a result, the dataset used in this study benefits from a refined evaluation process and improved editor training, ensuring higher reliability and consistency in the annotations used for training and evaluating the classification model.

2.3 Data Cleaning and Filtering

To prepare the dataset, the following steps were taken:

(i) Articles with fewer than 50 words were excluded to ensure sufficient content for classification.
(ii) Text was cleaned to remove emojis, HTML tags, and other non-text elements, focusing on title and main content.
(iii) Only articles were included, excluding other entity types like videos or images.

These measures ensured a clean and relevant dataset for journalistic quality classification.

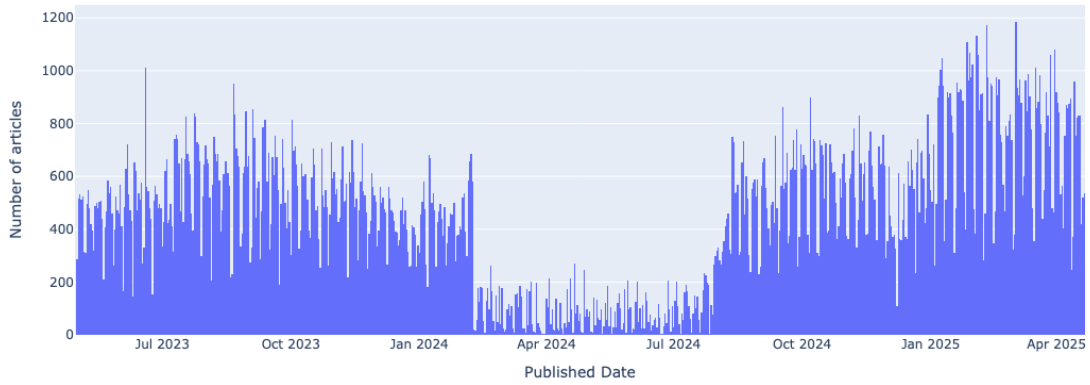


Figure 2.1: Number of labeled articles.

2.4 Annotation Process

The quality labels used as the ground truth in this supervised learning task were assigned by experienced human editors at Seznam.cz. This annotation process is a routine part of content management on the platform, aimed at maintaining consistent editorial standards.

Editors evaluate articles based on a set of internal guidelines, but also rely significantly on their professional judgment and experience. They assess various aspects, including depth of content, writing quality, factual accuracy, structure, sourcing, and overall adherence to journalistic principles. Based on this assessment, each article is assigned one of the four quality labels defined in Section 1.2:

Class 0 (Insufficient Articles)

Class 1 (Short but Sufficient Articles)

Class 2 (Standard Articles)

Class 3 (High-Quality Articles)

2.5 Dataset Statistics

Dataset Size and Splits

After cleaning and filtering, the data was split into three sets: Training Set, Validation Set and Test set. The distribution of articles across the four quality labels within each dataset split is presented in Tab. 2.1

Table 2.1: Class Distribution Across Dataset Splits

Set	Label 0	Label 1	Label 2	Label 3
Training	12,994 (22.9%)	32,086 (56.5%)	11,166 (19.7%)	542 (0.9%)
Validation	2,785	6,876	2,393	116
Test	2,795	6,863	2,404	118

As clearly visible from Tab. 2.1, the dataset exhibits significant class imbalance. The Short but Sufficient Articles constitutes the majority (over 56%) of the articles, while the

High-Quality Articles is extremely rare, representing less than 1% of the data. This imbalance presents a major challenge for model training, as standard algorithms tend to become biased towards the majority classes. Addressing this imbalance effectively is a key focus of the methodology described in the next chapter.

2.6 Data Analysis

Article length

Figure Fig. 2.2 shows the marginal distribution of article length in words for each class. The average article length is 545 words.

The distribution is strongly right–skewed: the shortest admissible texts contain 51 words, the median is 434 words and the longest article reaches almost 9000 words. This long tail already hints at a qualitative difference between routine news briefs and in-depth reporting.

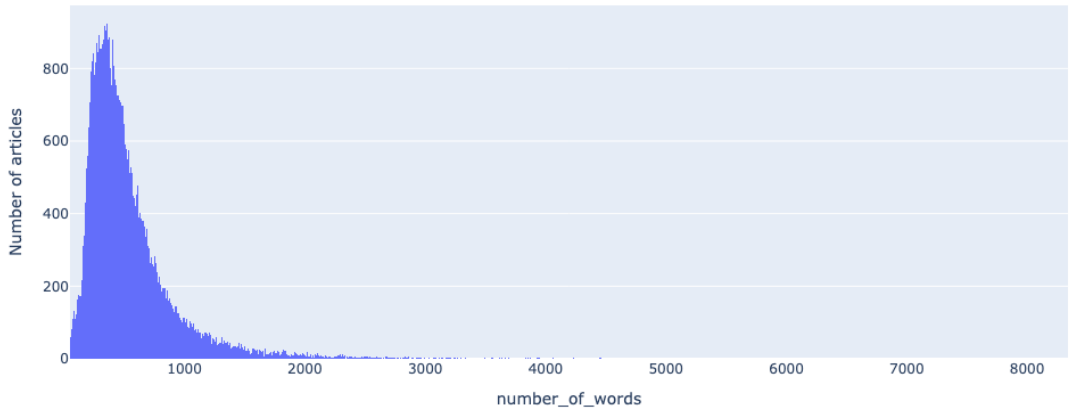


Figure 2.2: Distribution of article length in words.

Length by label

This Fig. 2.3 shows the distribution of article lengths for each quality label. The x-axis represents the number of words in an article, and the y-axis shows the probability density, so we can compare how article lengths differ across quality levels.

The results clearly show that better articles tend to be longer:

- Insufficient Articles and Short but Sufficient Articles articles are the shortest. Most of them are under 600 words, with label 1 forming the sharpest peak around 300-400 words.
- Standard Articles articles are longer, typically ranging from 400 to 1 200 words. They overlap with label 1 but clearly shift toward the right.
- High-Quality Articles articles are the longest. They have a wide distribution, with many texts over 1 500 words and some even exceeding 7 000 words. The long tail in the purple distribution reflects very detailed, in-depth articles.

This confirms that article length is a strong signal for quality. Longer texts are more likely to include detailed explanations, sources, and structured content—all features editors associate with high-quality journalism.

However, since the distributions overlap, especially between classes 1 and 2, article length alone is not enough to classify quality.

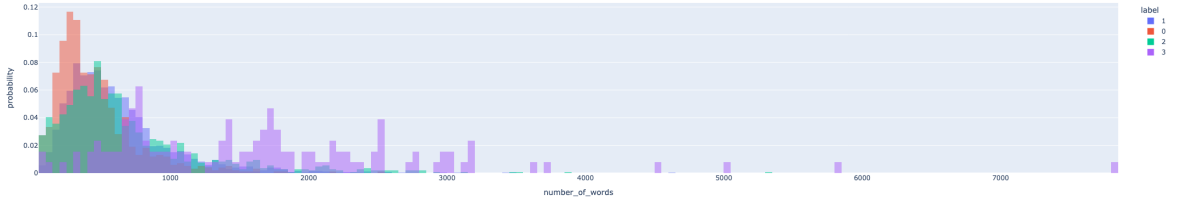


Figure 2.3: Distribution of article length in words by label

Unique-token probability density

The number of unique words in each article helps show how rich and varied the language is. Figure Fig. 2.4 shows that higher-quality articles usually have more unique words.

Insufficient Articles often have between 280 and 380 unique words and rarely go above 800. Most articles in Short but Sufficient Articles also stay under 500 unique words. Standard Articles articles use more words, and High-Quality Articles articles often have over 1000 unique words. Some even go above 3000.

This means that better articles tend to use a wider vocabulary. However, there is still a lot of overlap between the groups, especially for shorter texts. So, the number of unique words alone cannot perfectly tell the article quality but is still a useful signal. It works best when combined with other features like article length and sentence structure.

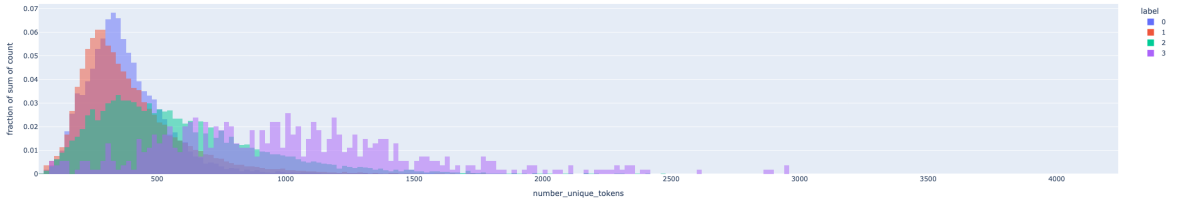


Figure 2.4: Distribution of unique tokens in articles.

2.7 Dataset Balancing Strategies

Data Augmentation

To increase the representation of Class 3 (the minority class), an attempt was made to augment the training data by including all available Class 3 articles from the full dataset, even those outside the original training period. This added approximately 2000 older articles to the training set. However, these older articles exhibited an outdated journalistic style that differed significantly from more recent content. Empirically, adding this data reduced the model's generalization ability: the validation F1 score dropped from 0.75 to 0.70. Due to this degradation in performance, the data augmentation approach was ultimately not used in the final model.

- **Oversampling**

Another strategy was random oversampling of Class 3 instances. In this approach, existing Class 3 articles were duplicated until their total count matched the number of Class 2 articles (around 11,000). Oversampling did improve the model's recall on Class 3 examples (approximately 0.30). However, it had significant drawbacks. Training time increased by roughly 20%, and the model began to overfit the training data. As a result, the validation F1 score fell to 0.68. Given these trade-offs, oversampling was not incorporated into the final training procedure.

- **Synthetic Data Generation**

A third approach involved generating new Class 3 samples using a text-generation technique. In practice, Class 1 articles were rewritten into High-Quality Articles in an attempt to synthesize additional Class 3 data. Unfortunately, the generated texts were not realistic and lacked the expected journalistic quality. The synthetic samples failed to resemble true Class 3 articles, and this approach did not improve model performance. Consequently, the synthetic data generation method was abandoned.

3 Evaluation Metrics

Evaluating the performance of a multi-class classification model for journalistic article quality requires careful selection of metrics, especially given the class imbalance in the dataset. As described in the Section 2.5, the four quality categories are not equally represented. This severe imbalance means that naive models could achieve seemingly high performance by favoring the majority class. Evaluation metrics must be chosen to reflect performance across all quality levels, rather than be dominated by the majority class.

3.1 Limitations of Accuracy

Accuracy measures the proportion of correct predictions out of all predictions. While it is a simple and intuitive metric, accuracy is inadequate for imbalanced datasets. In our context of quality classification, accuracy treats all prediction errors uniformly and thus can be misleading. A model labeling every article as Short but Sufficient Articles would achieve 54% accuracy by virtue of Class 1's prevalence, potentially outperforming more nuanced models that try to distinguish all four classes but incur some mistakes on the majority class. This illustrates a key limitation: **accuracy can be artificially high even if the model entirely ignores minority classes.**

For a journalistic quality assessment system, such behavior is unacceptable – correctly identifying the minority classes (e.g. truly high-quality pieces) is crucial. Therefore, relying on accuracy alone would not highlight a model's failure to detect rare yet important cases. In light of this, we turn to alternative metrics that better capture performance across all classes, giving due weight to both common and uncommon article categories. These metrics focus on the balance between types of errors, rather than just the overall fraction correct.

3.2 Precision

Precision is calculated as:

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{3.1}$$

where True Positive (TP) is the number of correctly predicted instances of a class, and False Positive (FP) is the number of incorrectly predicted instances where the model predicted the positive class but the actual label was negative.

Precision answers the question: "When the model predicts a certain class (especially a positive or high-quality outcome), how often is it correct?" In the context of our highest class (Class 3, high-quality articles), a high precision means that whenever the model flags an article as high-quality, it is usually correct. This is crucial for a platform like Seznam.cz because high precision in identifying top-quality content ensures that any article highlighted or promoted as "high-quality" truly meets the standards. In other words, high precision protects against false positives – it avoids promoting lower-quality articles as if they were premium content.

3.3 Recall

Recall measures the fraction of actual positive instances correctly identified by the model, calculated as:

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{3.2}$$

where False Negative (FN) is the number of actual positive instances that were incorrectly predicted as negative. It addresses "How many of the actual instances of a class did the model successfully find?" For High-Quality Articles, high recall means the model manages to identify most of the truly high-quality articles in the dataset. This is equally critical for our application: a high recall ensures that valuable content is not overlooked. In terms of business impact, high recall minimizes false negatives – it reduces the chance that an excellent article slips through the cracks and fails to be recognized or showcased as such. Missing a high-quality piece (a false negative) is a lost opportunity for user engagement and platform value, since exceptional content might not reach the audience it deserves.

3.4 F1 Score

The balance between precision and recall is a classic trade-off: improving one can often degrade the other. In our case, both are important. To illustrate:

High Precision – Ensures that the articles labeled as high-quality are truly valuable, avoiding the risk of promoting subpar content to users.

High Recall – Ensures that most high-quality articles are detected and not ignored, maximizing the coverage of valuable content.

Depending on the application needs, one might prioritize precision over recall or vice-versa. For example, if the platform strictly wants to avoid ever featuring a low-quality article as top-tier content, it would emphasize precision (even if some good articles are missed). Conversely, if the goal is to surface as many great articles as possible, it would emphasize recall (even if a few lower-quality ones are misclassified among them). In this thesis, however, we aim for a balance, since both types of errors have costs (as discussed later).

To evaluate model performance holistically, we employ the **F1 score**, which is the harmonic mean of precision and recall. The F1 score provides a balanced measure that only yields a high value when both precision and recall are high, making it well-suited for our imbalanced multi-class problem. It is defined as:

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.3}$$

This formulation accounts for the trade-off between false positives (FP) and false negatives (FN): a model cannot maximize F1 by excelling in only one of precision or recall at the expense of the other.

Unlike accuracy—which, as noted, treats all errors the same and can be dominated by the majority class—the F1 score ensures that performance on minority classes is not masked. A high F1 indicates the model is catching most of the high-quality articles (high recall) and not mislabeling too many low-quality articles as high-quality (high precision). In practical terms, F1 was used as the primary metric during model development: for example, the training procedure monitored the F1 score on the validation set and employed early stopping when it ceased improving , ensuring the final model is the one that best balances precision and recall.

3.5 Weighted F1 Score

When extending precision/recall/F1 to a multi-class setting, one must decide how to aggregate performance across classes. In this project we report the weighted F1 score as the primary evaluation metric, given the differing frequencies of each quality class. The weighted F1 is computed by calculating the F1 for each class separately, then taking a weighted average of these F1 scores using the class's support (proportion of samples) as weights. Formally:

$$\mathrm{F1W} = \sum_{i=1}^{n} (w_i \times \mathrm{F1}_i) \tag{3.4}$$

where $i \in \{0, 1, 2, 3\}$, $w_i = \frac{N_i}{N}$ is the fraction of samples belonging to class i in the dataset (i.e. its empirical probability) and $\mathrm{F1}_i$ is the F1 score for class i. This approach gives more influence to classes with more samples, while still incorporating performance on all classes.

Why use weighted F1?

In an imbalanced multi-class scenario like ours, weighted F1 offers several advantages over other averaging schemes (such as macro or micro averaging):

- Balances Class Influence: Macro-averaged F1 treats all classes equally, which can over-emphasize performance on very rare classes, while micro-averaging is dominated by the majority class (since it effectively counts every instance equally, making the aggregate metric similar to accuracy in a single-label setting). The weighted F1 is a compromise that considers each class's importance proportional to its frequency, preventing both the over-domination of majority classes and the over-sensitivity to tiny classes.
- Comprehensive Single Metric: The weighted F1 provides a single summary statistic that still reflects precision-recall balance across all categories. This is convenient for model selection and comparison. During development, we used the weighted F1 on the validation set to pick the best model and tune hyperparameters (see Methodology chapter on training procedure), since it ensured we were improving performance in an overall sense, not just for one favored class.
- Reflects Real Data: Weighted F1 aligns evaluation with the actual class distribution, ensuring that minority classes are considered without letting rare classes overly affect the result.

3.6 Confusion Matrix

The confusion matrix is a powerful tool for visualizing the performance of a multi-class classification model. It provides a detailed breakdown of how many instances from each true class were predicted as each possible class. This allows us to see not only the overall accuracy but also where the model is making specific errors. Example of Confusion Matrix in Fig. 3.1.

In summary, the weighted F1 score was deemed the most appropriate metric for this study, and it is the primary metric reported for our results. It directly influenced model training as well: for instance, the final model was selected based on the highest validation weighted F1, and training was stopped early once this metric stopped improving to avoid overfitting. By using weighted F1 consistently, we maintain evaluation consistency between training and testing phases in the face of class imbalance.

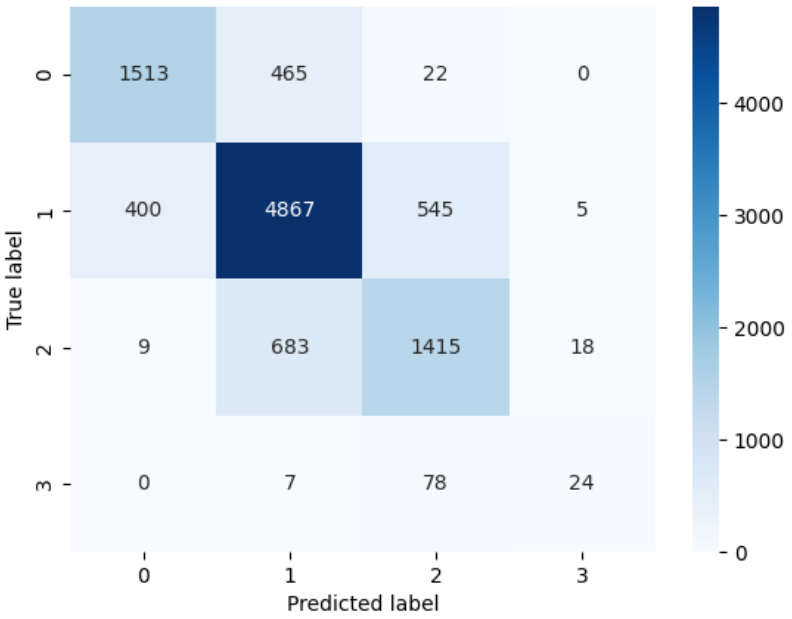


Figure 3.1: Example of Confusion Matrix. DebertaV3 Large.

3.7 Business Relevance

The choice of evaluation metrics is not only a technical consideration but also a business decision in the context of a platform like Seznam.cz. The ultimate goal of automating journalistic quality assessment is to have the model's predictions align with human editors' judgments across all quality categories, from Insufficient Articles up to High-Quality Articles. As described in the Methodology chapter, human editors currently provide the quality labels, and their agreement rate sets an upper bound on model performance. The metrics we choose should thus reflect how well the model replicates those editorial decisions.

In this regard, the weighted F1 score aligns well with business objectives. It measures the model's accuracy in mirroring human judgment while accounting for the prevalence of each quality category. In effect, a high weighted F1 indicates that the model is making the right distinctions between article qualities in roughly the same proportions as the real content mix on the platform. This is important for maintaining consistent quality standards: the platform seeks to reliably identify top-quality content and flag insufficient content, just as an editor would, but with far less manual effort. By using weighted F1 (which, as explained, ensures reasonable performance on each class), we directly support the goal that no class of article quality is neglected by the automated system. In business terms, this means the automation will uphold the overall editorial policy — it won't simply label everything as "average" to get a good accuracy, but will correctly spot the outstanding pieces and the sub-par ones according to their true frequency. A metric aligned with these needs helps guide model development towards an outcome that is valuable for the platform's content curation aims.

Cost of Classification Errors

In the context of content management, different types of classification errors carry distinct consequences, each affecting the platform in unique ways:

- **False Positives (FPs)** When lower-quality articles are incorrectly labeled as high-quality, they may be promoted as premium content. This can undermine user trust, as readers may feel misled by the platform's recommendations. Over time, such errors could harm the platform's reputation, making it less reliable in the eyes of its audience.
- **False Negatives (FNs)** Conversely, when truly high-quality articles are not recognized as such, the platform misses opportunities to showcase valuable material. This can reduce user engagement, as exceptional content fails to reach the audience it deserves, potentially limiting the platform's ability to retain and attract readers.

The F1 score addresses these challenges by striking a balance between precision (avoiding false positives) and recall (avoiding false negatives). This balance is crucial for managing the trade-off between promoting quality content and maintaining credibility. Additionally, the weighted variant of the F1 score ensures that less frequent categories—such as high-quality or insufficient articles—are not overlooked, despite their smaller presence in the dataset. By giving appropriate consideration to these minority classes, the metric supports a more comprehensive evaluation of the model's effectiveness, aligning with the platform's broader quality objectives.

4 Methodology

4.1 Editor Analysis and Upper Bound

Why Human Editors Are Important

When it comes to deciding what makes a good article, human judgment is key. Editors look at many factors, such as the depth of the content, the quality of the writing, the language used, and whether the article follows journalism standards. At Seznam.cz, experienced editors review articles to ensure they meet high standards. They follow clear guidelines but also rely on their own expertise. This human approach is valuable because it picks up on details and context that machines might miss. However, it's not perfect—it can be hard to stay consistent, it takes time, and it requires a lot of effort. Understanding how humans evaluate articles, including both their strengths and weaknesses, helps us design automatic systems that can do a similar job. Editors don't just focus on simple things like grammar or structure. They also check bigger issues, like whether the facts are correct, the story makes sense, and the journalism is fair and honest. But quality isn't always clear-cut—some aspects depend on personal opinions, so editors don't always agree. This is especially true for articles that aren't obviously great or terrible. These differences aren't a flaw; they just show how complex it is to judge quality.

How We Studied Editor Agreement

To understand how much editors agree and to set a target for automatic systems, we carried out a detailed study. We selected 100 articles from our dataset, choosing a mix of topics, dates, and quality levels to get a fair sample. Three experienced editors reviewed each article independently and sorted them into four categories. They followed the same rules used every day at Seznam.cz.

What We Learned About Editor Agreement

The results of our study were surprisingly strong—better than we expected. All three editors gave the same quality rating to 70 out of the 100 articles, which means a 70% full agreement rate. Even more impressive, at least two out of three editors agreed on the rating for every single article, giving us a 100% partial agreement rate. Interestingly, there was never a case where all three editors completely disagreed by picking totally different ratings. This tells us that while editors don't always see things exactly the same way, their disagreements are small—usually just between nearby categories, not about the article's overall quality. This shows a solid level of consistency among human experts.

Setting the Upper Bound

The 70% full agreement rate gives us a clear target, or "upper bound," for our automatic classification model. This is the level where even human experts fully agree, so it's a realistic goal for the system to aim for. It's also important to note that disagreements between editors were never drastic—they typically differed by just one category. This means that a reliable model should also aim to stay within this range. In practice, it's acceptable if the model's

predictions differ from human ratings by at most one category, since even experienced editors make mistakes of only ± 1 level. Ensuring this small margin of error helps maintain the system's trustworthiness and mirrors real-world human judgment.

4.2 Baseline Model Performance

Before developing complex models, we established baseline classifiers to provide lower-bound reference points. Baseline models are simple heuristics that do not incorporate any deep linguistic understanding, but they set minimum performance thresholds that any useful model should exceed. We implemented two naive strategies - a Random Classifier (Sec. 4.2.1) and a Most Frequent Class Classifier (Sec. 4.2.2) - and measured their effectiveness on the dataset. We report primarily the weighted F1 score as our performance metric.

Random Classifier

The Random baseline assigns a quality label to each article by sampling from the training set's class distribution. In other words, it ignores the article text and simply guesses a label at random, but with probabilities proportional to the observed frequencies of each class in the training data. This strategy produces an expected performance near chance level. Indeed, in our dataset - which is highly imbalanced (only 0.9% of articles are High-Quality, while the majority belong to the middle categories) - the random classifier achieved a weighted F1 of approximately 0.27. This very low F1 confirms that a classifier must do far more than random guessing to be useful. The random baseline underscores the challenge of the task: given the small fraction of high-quality articles and the dominance of the Short but Sufficient Articles, unguided guessing will mostly fail to identify the minority classes. This also highlights that any effective model needs to incorporate article-specific information (beyond class frequencies) to improve performance.

Most Frequent Class Classifier

The second baseline always predicts the single most common class from the training data for every article. In our case, the majority class was the Short but Sufficient Articles, which comprised a large portion of the dataset (Section 2.5 details the class distribution). By labeling every article as Short but Sufficient Articles, this classifier exploits the class imbalance to maximize overall accuracy, while entirely ignoring content. Perhaps surprisingly, this simplistic strategy achieved a weighted F1 of around 0.43, significantly higher than the random baseline. The reason is that by always predicting the majority class, it correctly "classifies" all truly Short but Sufficient Article (recall for Short but Sufficient Articles is 1.0), which carry a lot of weight in the weighted F1 calculation. This result demonstrates how a skewed dataset can let a trivial predictor attain a deceptively decent aggregate score. However, the limitations of the most-frequent-class model are clear: it fails to recognize any minority class (Precision and Recall are 0 for minority classes). It cannot differentiate the subtle cues that distinguish high-quality journalism from mediocre content.

Discussion

The results of these baseline models shed light on key aspects of our classification challenge. The Random Classifier's minimal performance underscores the necessity of incorporating article-specific information to achieve meaningful outcomes, rather than depending on random predictions. In contrast, the Most Frequent Class Classifier's score of 0.43 demonstrates the pronounced effect of class imbalance, as it achieves a respectable result solely by leveraging the majority class. These baselines set clear standards for evaluating future models. Any advanced classifier—whether employing intricate feature engineering or sophisti-

cated machine learning techniques—must exceed these initial benchmarks. Beyond improving overall metrics, we seek models that excel at correctly classifying less frequent yet significant quality categories. This establishes a critical goal for further development: to create systems that capture the nuanced elements of journalistic quality and more closely align with human editorial assessments.

4.3 Feature Engineering

The goal of the feature engineering phase is to translate each article into a set of quantitative attributes that reflect its journalistic quality. We design these features using principles from computational linguistics and journalism theory to capture various aspects of writing and content. This allows the classification model to assess quality based on interpretable characteristics of the text. (Certain implementation details are omitted here due to confidentiality, but the underlying methods and rationale are described.) The features chosen are grounded in four key dimensions of journalistic quality – **Diversity**, **Transparency**, **Attractiveness**, and **Independence** – ensuring that our approach aligns with established editorial standards and ethical AI practices. In other words, each feature group corresponds to one or more of these quality dimensions, so that the model's decisions are based on meaningful, fair criteria rather than arbitrary signals. The following subsections detail each feature group and its relation to the quality dimensions.

Linguistic Complexity Features (Attractiveness)

One important aspect of article quality is the richness and complexity of its language. Linguistic complexity features quantify the sophistication of writing, which corresponds to the Attractiveness dimension of quality – an engaging, well-crafted article typically uses varied and well-structured language. In the Czech language context, we pay special attention to syntax, vocabulary variety, and morphology. This group includes:

- **Syntactic Complexity:** Measures of sentence structure complexity, such as average sentence length and the use of subordinate clauses or compound sentences. High-quality articles often contain longer, information-rich sentences or a mix of sentence lengths that provide detail and nuance. In contrast, lower-quality pieces may rely on very short, simple sentences. We capture this by computing the average number of words per sentence and identifying complex sentence constructions. (For Czech, we note that free word order can make sentences longer or differently structured without losing clarity, so we focus on the presence of well-formed clauses and punctuation rather than word order alone.)
- **Lexical Diversity:** Metrics that assess the variety of vocabulary used, most notably the type-token ratio – the proportion of unique words (types) to total words (tokens) in the article. A higher unique word ratio indicates a broader vocabulary and more varied word choice, which is often a sign of a more informative and engaging article. As shown by the data analysis in Section 2.6, higher-quality articles tend to employ more unique words than lower-quality ones. For instance, Figure 2.4 in Section 2.6 illustrates that High-Quality Articles often contain well over 1,000 unique tokens, whereas Insufficient Articles have far fewer unique words. This confirms that better articles use a wider vocabulary, though there is still some overlap between classes.
- **Morphological Richness:** Features that capture the range of word forms and complex words. Czech is a highly inflected language, meaning words change form (through prefixes, suffixes, case endings, etc.) and new words can be formed via derivation and compounding. We account for derived word forms and compound nouns usage in the text. A quality article might use more varied word forms (indicating precise expression and nuanced meaning) and appropriate compounds, reflecting a command of language. In summary, a high morphological variety signals that the author is leveraging the language's full expressive capability, aligning with an attractive writing style.

Together, these linguistic complexity features describe how sophisticated and varied the article's language is. They tie into the Attractiveness dimension because an article that is written with complex sentence structures, a broad vocabulary, and rich morphology is generally more engaging and indicative of higher journalistic effort. However, we also remain mindful that clarity is important – complexity should not come at the expense of understanding. (This balance is addressed by the readability features, described later.) Notably, initial data exploration showed that these features are promising: for instance, articles that editors rated as high quality exhibited significantly longer sentences and greater vocabulary variety than low-quality ones, which justifies including these metrics.

- **Structural Composition Features (Transparency)**

Another key aspect of quality is how the article is organized and whether it adheres to good journalistic practices in its structure and use of sources. Structural composition features describe the layout and content elements of an article beyond just the raw text. These features align mostly with the Transparency dimension (and also support Independence), as they reflect how openly and clearly the article presents information and sources. High-quality journalism is usually well-structured and transparent about where information comes from. We include several features in this group:

- **Article Length:** The total length of the article (measured in words). This is a fundamental structural attribute – while not a quality guarantee on its own, length often correlates with depth of content. In Section 2.6, we observed that better-quality articles tend to be much longer on average. Figure 2.3 in Section 2.6 shows the distribution of article lengths by quality class: High-Quality Articles have a wide length distribution with many articles over 1,500 words (and some even above 5,000 words), whereas Insufficient Articles are mostly under 600 words. This indicates that longer texts are more likely to include detailed explanations, background, and evidence – traits of high-quality reporting.
- **Section Organization:** Features capturing how the article is structured into sections or paragraphs. A well-structured news article typically has a clear introduction (lead), a body divided into coherent paragraphs or sub-sections, and sometimes a conclusion. We quantify this by looking at the number of paragraphs or explicit sections, the presence of subheadings or section titles, and the overall flow. For example, an article with many one-sentence paragraphs or a jumbled structure might indicate lower quality or a rushed piece, whereas one with logically separated sections (e.g. an introduction, several thematic paragraphs, and a wrap-up) suggests careful composition. This structural awareness contributes to transparency and readability – an article that is organized logically is easier for readers to follow and shows that the author/editors put thought into presenting the information clearly.
- **Multimedia Content Metadata:** Considers the presence and originality of multimedia elements like images, videos, and infographics. Original, journalist-taken photographs significantly enhance transparency, distinguishing higher-quality journalism from stock images. Additionally, metadata such as image descriptions or alt-text indicate editorial commitment to accessibility and clarity.
- **Citations and Source References:** Metrics evaluating how the article cites sources or includes references. A transparent article will often mention where information comes from – for example, quoting officials, referencing reports, or providing hyperlinks to source documents. We also consider source diversity in a basic way: if an article cites

several different people or organizations, it's likely providing a more balanced, independent view (related to the Independence dimension as well). This encourages the classifier to reward articles that follow the journalistic norm of attributing information, aligning with higher quality. Also we classify quotes by origin—whether sourced directly by the journalist (original quotes), from agencies, directly spoken to the channel, or indirectly through platforms like social media (Instagram). Clearly attributed original quotes signal high transparency and credibility.

Overall, the structural composition features assess whether an article is well-built in terms of format and sourcing. These features reflect the Transparency dimension because they deal with how openly the article presents its information structure and sources to the reader. A well-structured article with clear sections, supporting images, and cited sources demonstrates transparency and professionalism. These are qualities we expect in higher quality categories. Additionally, by considering structural elements, we also indirectly support the Independence dimension: for example, using multiple sources and proper citations suggests independent verification of facts rather than just copying a single source. The combination of these structural features helps the model distinguish a thoroughly crafted article from a shoddy one on more than just vocabulary – it looks at the article's journalistic form.

- **Readability Metrics**

While the previous features focus on complexity and content, it is also important that a high-quality article is readable and accessible to its intended audience. To capture this, we include readability metrics that gauge how easy or difficult a text is to read. These features complement linguistic complexity by ensuring that we don't just reward complexity for its own sake – the writing should also be clear. In other words, Attractiveness in journalistic quality is a balance between rich content and reader-friendly presentation.

We adapt established readability formulas for the Czech language, accounting for Czech's morphological complexity and free word order. Classic readability measures were originally developed for English, using factors such as sentence length and syllable count per word. However, applying them directly to Czech can be misleading because Czech words tend to be longer (due to inflectional endings) and the sentence structure can be more flexible. Therefore, we calibrate these formulas with Czech-specific considerations. Research has shown that a Russian-adapted formula works better for Czech than the raw English one, due to similarities in language structure, so we base our adaptation on similar principles (given the closeness of Slavic languages). In practice, this means our readability score might weight sentence length a bit more heavily (since very long sentences can be hard to parse in Czech) and adjust the baseline expectations for word length and syllable count.

The outcome is a numeric readability score for each article. A higher score (or lower difficulty) indicates the article is easier to read, while a lower score means it's more complex or dense. We expect high-quality articles to fall in a moderate range: they should not be trivial to read (as overly simplistic text might indicate shallow content), but they also should not be needlessly convoluted. By including the readability metric, we ensure the model can penalize articles that are linguistically complex but unreadable, and conversely recognize when a slightly simpler writing style still conveys high-quality information effectively.

In summary, the readability feature provides a check on the Attractiveness dimension: it helps distinguish content that is both rich and well-written from content that is rich but poorly presented. This contributes to a more nuanced quality assessment, aligning our feature

set with human perceptions of what makes an article good – not just what is written, but how it comes across to the reader.

- **Metadata Features (Diversity)**

Articles don't exist in isolation — contextual information about an article can also relate to its quality. In this context, Diversity refers to capturing a broad range of content conditions and ensuring the model is aware of different sources and patterns, so it can handle the variety present in the dataset.

Channel ID (Source Identifier): Each article in our dataset comes from a specific source or channel (for example, a particular news outlet or a section of the news platform). Initially, we included an anonymized channel identifier as a metadata feature. This allowed the model to recognize patterns in writing style or typical quality that might differ between various channels. Although this feature significantly improved classification performance, we ultimately decided not to use it in our final model.

The primary reason for excluding the channel identifier was to ensure the model classifies the articles based solely on textual content rather than relying on the reputation or historical patterns of specific channels. If we had used the channel ID, the model would strongly overfit to these identifiers—producing artificially high results by learning channel-specific biases. Consequently, if a particular channel aimed to improve its journalistic quality, the model might fail to reflect these improvements accurately, as it would still base predictions on outdated channel-specific patterns rather than on the actual article quality.

Furthermore, relying on channel identifiers could cause problems when classifying articles from channels that did not appear in the training dataset. In such cases, the model would lack critical information about these unseen channels, reducing its predictive reliability. By excluding this feature, we ensure the model remains generalizable and sensitive to the actual text content, facilitating a fair and unbiased assessment of article quality across all channels.

4.4 Performance of Pre-trained Language Models

Transformer-based language models have revolutionized Natural Language Processing (NLP) across multiple languages, including Czech. This chapter examines the theoretical foundations, architectures, and practical applications of pre-trained language models for Czech text classification tasks, with special attention to models specifically designed for the Czech language and their performance characteristics.

Theoretical Foundations

Pre-trained language models mark a significant advancement in NLP by shifting from traditional word-level representations to models capable of capturing nuanced contextual information. These models are built upon the transformer architecture, first introduced by Vaswani et al. 2017, which relies on self-attention mechanisms to model relationships across entire sequences. The architecture, illustrated in Figure 4.1, enables highly parallelizable computation and has become foundational for state-of-the-art models in tasks such as classification, information retrieval, and text generation.

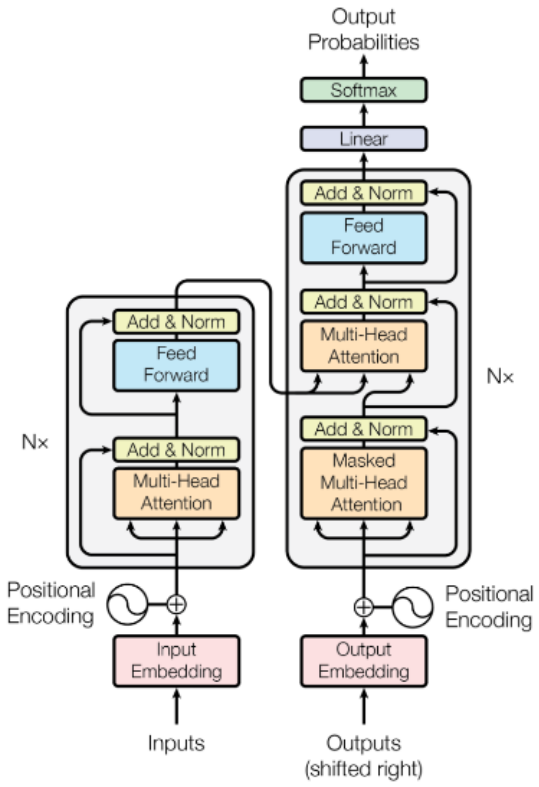


Figure 4.1: Overview of the Transformer architecture, as introduced in the paper *Attention is All You Need* Vaswani et al. 2017.

Evaluation of Multiple Pre-trained Models

For this research, four pre-trained Bidirectional Encoder Representations from Transformers (BERT)-like (Devlin et al. 2019) language models developed internally at Seznam were evaluated. The models - Base-sized Roberta using Wordpiece tokenization for Czech

(BROWCzech), Retrieval-Oriented Masked Autoencoder (RetroMAE), Decoding-enhanced BERT with Disentangled Attention (DeBERTaV3) Base, and DeBERTaV3 Large - were fine-tuned to predict quality ratings for articles.

BROWCzech

Architecture BROWCzech is a monolingual Czech language model developed by Seznam.cz. It is based on the RoBERTa (Liu et al. 2019) architecture and was pre-trained on an extensive internal corpus of Czech webpages, encompassing diverse topics and writing styles. This extensive pre-training enables the model to capture the nuances of the Czech language effectively.

- Parameters: 109.5 million (23.4M in the embedding layer + 86.0M in the transformer layers)
- Layers: 12 transformer layers
- Hidden Dimensions: 768
- Input Tokens: Maximum of 512
- Tokenizer: Monolingual, supporting only Czech

Pre-Training The model was trained using the Masked Language Modeling (MLM) objective, where a portion of the input tokens is masked, and the model learns to predict these masked tokens. This training approach allows the model to develop a deep understanding of the language structure and semantics. The pre-training corpus consisted of a vast collection of Czech web pages, ensuring a comprehensive representation of the language used in various contexts.

Special Capabilities Due to its monolingual focus and extensive pre-training on Czech data, BROWCzech excels in tasks requiring a deep understanding of the Czech language. It is particularly effective in scenarios where the text is exclusively in Czech, providing high-quality contextual embeddings that enhance performance in downstream tasks such as text classification and sentiment analysis.

RetroMAE

Architecture RetroMAE (Xiao et al. 2022) is a transformer encoder that integrates a specialized pre-training architecture for better document-level representations. It introduces an asymmetric encoder-decoder setup during pre-trainin. The encoder is a full-scale BERT-like transformer which produces a condensed vector representation of the input text (for instance, the [CLS] token embedding). The decoder is a lightweight one-layer transformer that uses this representation to reconstruct the original text. This design does not change the model's structure at fine-tuning time - we still fine-tune the 12-layer encoder on the classification task - but the pre-training setup encourages the encoder to act as a powerful sentence/document encoder. RetroMAE uses a Czech tokenizer, so in architecture it is comparable to BROWCzech aside from the pre-training apparatus.

- Parameters: 130.0 million (43.9M in the embedding layer + 86.0M in the transformer layers)
- Layers: 12 transformer layers
- Hidden Dimensions: 768
- Input Tokens: Maximum of 512
- Positional Embedding: absolute

- Tokenizer: Monolingual, supporting only Czech

Pre-Training RetroMAE was pre-trained on a massive Czech text corpus (≈253 GB of news and web articles) with a novel masked autoencoding strategy geared towards retrieval. The key idea is to train the encoder to produce embeddings that capture the meaning of the entire text, which can be used to regenerate the text. To achieve this, RetroMAE applies different masking ratios to the encoder and decoder inputs: for each input, the encoder sees a moderately masked version (20% tokens removed) while the decoder sees a heavily masked version (50% tokens masked out). The encoder must output a representation that, when fed into the decoder along with the decoder's masked input, allows the decoder to reconstruct the original sentence (recover the masked tokens). Additionally, RetroMAE employs an asymmetric model structure, with the encoder being a full transformer (to encode rich features) and the decoder being minimal (to force the burden of representation onto the encoder). This pre-training approach effectively teaches the encoder to compress each document's essential content into its embedding. Beyond autoencoding, RetroMAE's training in this project also included retrieval-based contrastive learning: the model was periodically tasked to bring embeddings of similar documents closer and push dissimilar ones apar. This two-pronged pre-training (reconstruction + contrastive objectives) leads to embeddings that are not only information-rich but also arranged in a semantic space useful for comparing texts. Indeed, the creators of RetroMAE report that it dramatically improves performance on retrieval benchmarks, confirming that the encoder learns powerful semantic representations.

Special Capabilities Thanks to its training, RetroMAE excels at capturing document-level semantics and topical depth. It effectively encodes the "aboutness" of an article - the model was literally trained to compress an article's meaning into one vector. For journalistic quality, this means RetroMAE can distinguish content-rich, in-depth articles from superficial ones. For example, an article with thorough coverage of a subject (lots of pertinent details, background context, and related information) will produce a dense embedding that RetroMAE recognizes as distinct from that of a short news brief. This strength aligns with the "depth" dimension of quality: RetroMAE should be adept at flagging articles that lack substance (likely classifying them as lower quality) versus those with comprehensive coverage (higher quality). Its retrieval objective may also enhance detection of factual consistency - articles that cover similar events should have similar embeddings, so if an article's content is anomalous or off-topic (potentially indicating lower factual accuracy or relevance), RetroMAE might more readily identify it. On the other hand, RetroMAE does not introduce special handling of language beyond Czech. It might not handle foreign names or code-switching as well as a bilingual model, and it relies on the same 512-token input limit as BROWCzech. Also, because its focus is on what is said (semantic content) more than how it's structured, RetroMAE might be less sensitive to certain stylistic or structural quality signals (e.g., subtle differences in tone or the presence of formatting elements like citations). In practice, however, document embeddings can encode some structural indicators if they correlate with content. Overall, RetroMAE's semantic prowess is expected to help on quality aspects like factual depth and topic relevance, complementing other models that emphasize linguistic or structural nuances.

DeBERTaV3 Base

Architecture DeBERTaV3 (He, Gao, and Chen 2021) is a transformer encoder that builds on the BERT family with two major innovations: disentangled attention and improved embeddings. In DeBERTaV3, each token is characterized by two vectors - one for its content

(token identity) and one for its position - instead of a single combined embeddin. During self-attention, the model can attend to content and position information separately, a mechanism known as **Disentangled Attention**. This enables a form of relative position encoding: the model doesn't just rely on fixed absolute positions, but learns relative positional biases so it can better handle long-range dependencies and varied word order. The Base model also employs an extended vocabulary (in our case, a bilingual Czech-English tokenizer) which increases the embedding layer parameters. The bilingual tokenizer and vocabulary allow it to natively process both Czech and English text. This is particularly useful for Czech news, which often contain English names, technical terms, or quotes. In summary, DeBERTaV3 Base's architecture offers a more expressive attention mechanism and cross-lingual lexical coverage, setting it apart from BROWCzech and RetroMAE which use standard attention and monolingual vocabularies.

- Parameters: 129.4 million (43.9M in the embedding layer + 85.5M in the transformer layers)
- Layers: 12 transformer layers
- Hidden Dimensions: 768
- Input Tokens: Pre-trained with sentences with max 512 tokens, generalizes up to 2048 due to relative position embeddings
- Tokenizer: Bilingual, supporting both Czech and English

Pre-Training DeBERTaV3 Base was pre-trained with a state-of-the-art strategy that combines ideas from Efficiently Learning an Encoder that Classifies Token Replacements Accurately (ELECTRA) and DeBERTa. Instead of the traditional MLM objective, it uses Replaced Token Detection (RTD) (Clark et al. 2020) - the model is trained to distinguish real tokens from plausible but fake tokens generated by a small generator network. This task is more sample-efficient than MLM, yielding better learning of token distinctions. Additionally, DeBERTaV3 introduced Gradient-Disentangled Embedding Sharing, a technique to allow the generator and discriminator (in the ELECTRA setup) to share token embeddings without conflicting gradients (avoiding a "tug-of-war" in embedding updates). By applying RTD with disentangled embeddings, DeBERTaV3 achieves faster and more effective pre-training than MLM-based model. The model was trained on a mixture of English and Czech text (a bilingual corpus), reflecting its dual-language design. This means it saw Czech news articles and possibly English texts during pre-training, enabling it to handle code-switching and borrowed foreign phrases. The disentangled attention from the original DeBERTa (He et al. 2020) is retained, meaning the model also learns relative position representations during pre-training. Overall, this advanced pre-training regimen endows DeBERTaV3 Base with both multilingual understanding and robust language modeling, which have been shown to boost performance on a wide range of NLP tasks and even cross-lingual benchmark.

Special Capabilities DeBERTaV3 Base brings together several capabilities that are particularly relevant for assessing journalistic quality. First, its bilingual nature ensures that the presence of English names or excerpts in a Czech article will not confuse the model - it recognizes those tokens and their meanings. This supports the transparency dimension, as articles often mention international organizations or use English technical terms; DeBERTa can comprehend these in context and contribute to a fair quality assessment. Second, the disentangled attention and relative positional encoding give DeBERTa an edge in understanding text structure and long-distance relationships. News articles have important structural elements (e.g., key facts at the beginning, quotes and references throughout). DeBERTaV3 can better model such patterns, for instance understanding that a statement followed by a dash and a name is likely a quote attribution - a subtle structural cue linked to transparency and

credibility. It is also less likely to be thrown off by long sentences or paragraphs, since relative positioning helps it maintain coherence over longer spans. These abilities align with the structure and coherence aspect of quality: DeBERTaV3 might more accurately evaluate whether an article is well-organized and whether information is appropriately placed. Third, the model's ELECTRA-style pre-training (RTD) tends to sharpen its ability to catch anomalies or errors in text (since it learned to spot "implausible" tokens). This could translate into better detection of factual errors or inconsistent details, aiding the factual accuracy dimension. In essence, DeBERTaV3 Base is expected to perform strongly across multiple quality criteria - it handles language diversity, captures long-range context, and has a finely-tuned sense for language correctness. Its main limitation relative to its Large counterpart is capacity: as a base model, it may sometimes miss very subtle signals that a larger model could pick up, and it shares the same 512-token input limit (mitigated by relative encoding, but still in effect). Nonetheless, among base-sized models, DeBERTaV3 Base is the most advanced in this lineup.

DeBERTaV3 Large

Architecture The Large variant of DeBERTaV3 significantly scales up the Base model, enhancing its capacity for handling nuanced and complex language tasks. It is specifically optimized for scenarios requiring deeper semantic understanding and improved performance in demanding classification and cross-encoder tasks.

- Parameters: 361.4 million (58.6M in the embedding layer + 302.8M in the transformer layers)
- Layers: 24
- Hidden Dimensions: 1024
- Input Tokens: pre-trained with 512 tokens, generalizes up to 2048 via relative position encoding
- Vocabulary Size: bilingual Czech-English

Pre-Training Unlike the Base variant, DeBERTaV3 Large is trained with an increased model capacity, allowing it to capture subtler language nuances and richer contextual relationships. While maintaining the ELECTRA-style RTD objective, its larger model size provides more parameters to effectively differentiate authentic and artificially generated tokens, resulting in improved learning efficiency and deeper semantic representations. The training corpus remains bilingual, similar to the Base variant, but the enhanced parameter count enables superior assimilation of cross-lingual contextual clues and more robust modeling of long-form and nuanced texts.

Special Capabilities Compared to DeBERTaV3 Base, the Large model demonstrates markedly improved performance in tasks requiring detailed semantic comprehension and fine-grained textual distinctions. Its increased number of transformer layers and expanded hidden dimensions enhance its ability to resolve subtle language patterns, ambiguous contexts, and complex factual consistency scenarios. The larger model is better suited for detailed anomaly detection tasks, such as identifying intricate factual errors or subtle inconsistencies. Additionally, its superior long-range modeling capability due to enhanced relative positional encoding makes it particularly effective in evaluating coherence and structural quality of lengthy articles or documents. This positions DeBERTaV3 Large as the preferred choice for high-stakes journalistic quality assessments and intricate text classification tasks, surpassing the Base variant in overall accuracy and contextual sensitivity.

Model Selection

To compare the performance of the evaluated models, we trained them using the same loss function (described in Section 4.6), identical features, and the same context window size. Hyperparameters for each model were carefully tuned to achieve the best possible validation performance. In Table 5.1, we present the weighted F1 scores and the F1 scores for each quality class ($F1_1$–$F1_4$) on the test set.

Model	**$F1_1$**	**$F1_2$**	**$F1_3$**	**$F1_4$**	**F1 Weighted**	**Accuracy**
BROWCzech	0.727	0.752	0.656	0.359	0.711	0.70
RetroMAE	0.725	0.749	0.656	0.299	0.721	0.72
DeBERTaV3 Base	0.727	0.752	0.656	0.359	0.724	0.73
DeBERTaV3 Large	0.75	0.78	0.66	0.27	0.740	0.738

Table 4.1: Validation F1 scores for each model. $F1_1$–$F1_4$ correspond to classification quality for each label class. Accuracy is the overall accuracy of the model on the validation set.

DeBERTaV3 Large achieved the best weighted F1 score on the validation set, indicating that it is better suited for the quality classification task compared to the other models. Despite all models being trained under the same conditions, the increased model capacity and architectural improvements of DeBERTaV3 Large allowed it to generalize more effectively on unseen data.

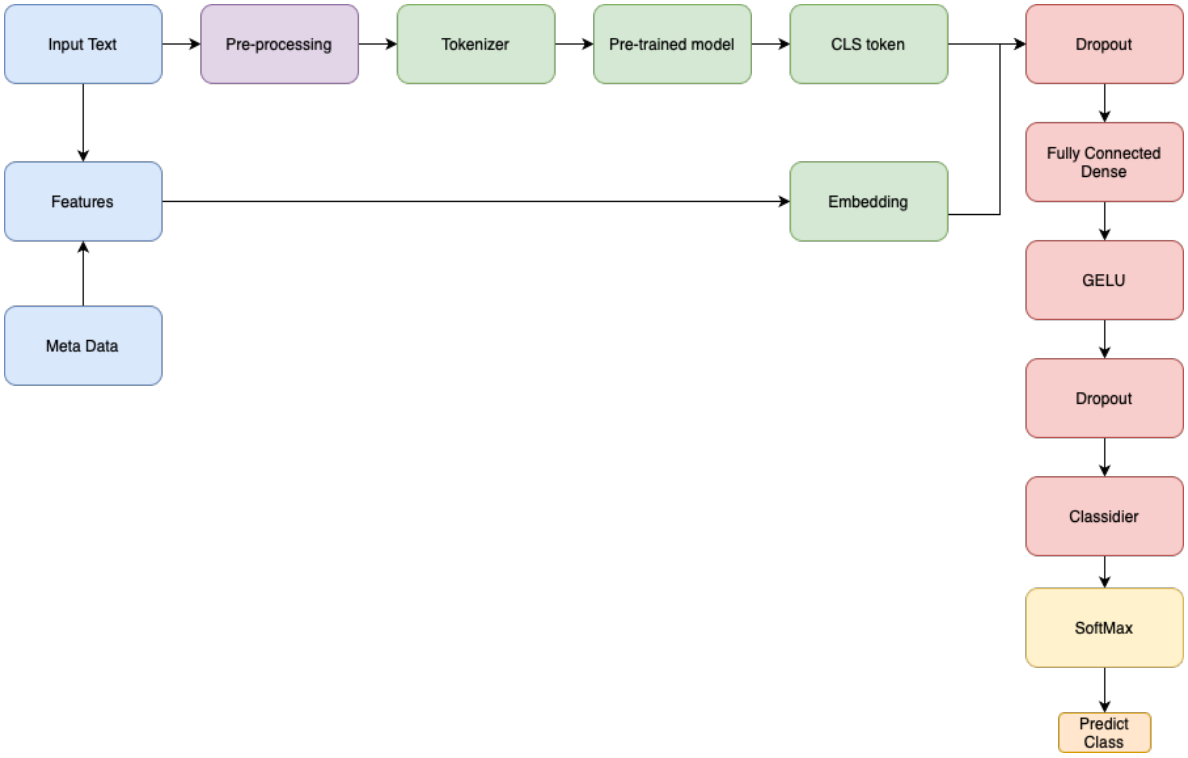


Figure 4.2: Model architecture overview for journalistic quality classification.

4.5 Classifier Architecture Design

The classification architecture implemented in this research was designed to efficiently leverage the semantic information extracted by large pre-trained language models while introducing minimal additional complexity to prevent overfitting, particularly given the class imbalance of the dataset.

Initially, during experiments with the BROWCzech model, a simple classification head was developed, consisting of a dropout layer, a dense projection layer, and a softmax output. However, through systematic experimentation, it became evident that a slightly deeper and regularized architecture led to improved performance without significant computational overhead. These improvements, although first tested with BROWCzech, were later adapted and finalized in conjunction with the DeBERTaV3 model, which ultimately demonstrated superior performance and became the backbone of the final system.

The overall data flow and model architecture are illustrated in Figure 4.2 As shown, the input text is first pre-processed and tokenized, then passed through the pre-trained transformer model to obtain contextual embeddings. The [CLS] token embedding is processed through a regularized fully connected classifier head, including dropout, dense projection, Gaussian Error Linear Unit (GELU) (Hendrycks and Gimpel 2016) activation, and another dropout, before producing the final logits for prediction. Separately, additional features and metadata were extracted for possible use in ensemble models (described in Section 4.3).

4.6 Loss Function Engineering

Designing an effective loss function was critical due to the ordinal nature of the quality labels and the severe class imbalance in the dataset. The development of the loss function progressed through several stages, from a baseline categorical Cross-Entropy Loss (CE) to a final hybrid loss with an ordinal component and class weighting. Each refinement aimed to address specific shortcomings observed in earlier trials.

Initial Approach: Cross-Entropy Loss

The initial model training used the standard categorical CE loss, given its widespread success in multi-class classification. The cross-entropy loss is defined as:

$$L_{\text{CE}} = -\frac{1}{N}\sum_{i=1}^{N}\log(\hat{p}_{i,y_i}), \tag{4.1}$$

where N is the number of samples in batch, for training sample i, y_i is true class label and $\hat{p}_{i,c}$ is predicted probability for each class c. This loss encourages the model to assign high probability to the correct class. CE was chosen as the starting point because it generally converges faster and more reliably than regression losses for classification problems (due to its strong gradients when the predicted probability is low for the true class). In early experiments, the CE loss yielded reasonable overall accuracy. However, closer inspection revealed two major issues: it **treated all misclassifications equally**, and it **struggled with the extreme class imbalance in the data.**

Shortcomings of Using CE Alone

Ordinal Errors Treated Equally: In our task, the four quality classes have an inherent order (from Insufficient Articles up to High-Quality Articles). A misclassification that is off by several levels (e.g. predicting class 0 for an article that is truly class 3) is far more severe than a misclassification to an adjacent class. Yet, standard cross-entropy provides no mechanism to differentiate these cases - it penalizes any incorrect prediction based only on the probability assigned to the true class, not how far off the predicted class is. In practice this meant the model might treat a confusion between class 3 vs 0 as no worse than between class 3 vs 2, despite the former being a much larger error in an ordinal sense. Such behavior is undesirable for quality assessment, where we want to strongly discourage large deviations in predicted quality. This limitation is well-recognized in ordinal classification literature: common classification losses "do not account for any distance information between prompting the introduction of specialized ordinal losses that penalize distant misclassifications more heavily.

Class Imbalance: Moreover, the dataset was highly imbalanced. Using plain cross-entropy, the model tended to optimize for the majority classes and could virtually ignore the minority class, since mis-predicting the rare class had minimal impact on the average loss. Indeed, an initial model with unweighted CE practically never predicted the High-Quality Articles, leading to an extremely low recall for that category (the model would default to predicting more frequent classes to minimize loss). This is a common outcome in imbalanced data settings, where the loss is dominated by the prevalent classes. Thus, additional measures were needed to ensure the model learned to recognize the minority class and to account for the ordinal scale of errors.

Addressing Class Imbalance: Focal Loss vs. Class Weighting

While the CE loss served as a strong initial baseline, we still needed to tackle the class imbalance so that minority classes (especially High-Quality Articles) would be properly recognized. Two strategies were explored: **Focal Loss** and **class weighting** within the cross-entropy.

Focal Loss Attempt

Focal loss was considered because it is specifically designed by Lin et al. 2017 to address class imbalance by down-weighting easy, majority-class examples. Focal loss adds a modulating factor $(1-p)^\gamma$ to the standard CE loss, so that well-classified examples (with high p for the true class) incur much smaller loss, focusing training on the hard, misclassified examples. The focal loss (without class-specific weighting) is:

$$L_{\text{FL}} = -\frac{1}{N}\sum_{i=1}^{N}(1-\hat{p}_{i,y_i})^\gamma \log(\hat{p}_{i,y_i}), \tag{4.2}$$

where $\gamma \geq 0$ is the focusing parameter. In theory, this loss would make the model devote more attention to the High-Quality Articles, since those examples were consistently mispredicted (i.e. $\hat{p}i, y_i$ for class 3 was often low, making $(1-\hat{p}i, y_i)^\gamma$ large and thus boosting their loss contribution). Focal loss has seen success in extreme imbalance settings (e.g. one-stage object detectors where background examples vastly outnumber foreground objects).

Outcomes with Focal Loss

In our setting, however, focal loss did not yield the desired improvements. The model trained with focal loss showed lower overall F1 than the simpler weighted loss approach (described in Section 4.6.3), and it especially underperformed on the minority class. Several factors may explain this result.

First, the imbalance in our data, while significant, is not as astronomical as in some detection problems - thus the benefit of focal modulation was less pronounced. Second, focal loss can implicitly down-weight not just easy majority-class examples but also easy examples of all classes; this means some well-classified minority instances might receive less focus than they deserve. In practice, we found that the model still struggled to identify High-Quality articles with focal loss - presumably because those samples were extremely rare to begin with, and focal loss alone cannot increase their presence in the training signal enough. Tuning γ did not reverse this trend, and the added complexity of focal loss did not justify itself for our problem. Similar observations have been noted in literature when using focal loss outside its original context - if the class imbalance is better handled by explicit re-weighting, focal loss can sometimes even degrade performance by over-focusing on a handful of difficult cases and under-training on the rest. Given these findings, we turned to a more direct approach: class-weighted loss, which ultimately proved more effective.

Class Weighting

The final loss function incorporated class weights into the cross-entropy term to directly counteract the imbalance. We assigned a weight w_c to each class c in the cross-entropy calculation, so that errors on minority classes are amplified and errors on majority classes are

down-weighted. The weights were set inversely proportional to class frequencies (with some manual tuning). In the end, the chosen weights were:

- $w_0 = 1.0$ for class 0: Insufficient Articles
- $w_1 = 0.5$ for class 1: Short but Sufficient Articles
- $w_2 = 1.0$ for class 2: Standard Articles
- $w_3 = 20.0$ for class 3: High-Quality Articles

These values mean, for example, that misclassifying a High-Quality Articles incurs 20× the loss of misclassifying a Standard Articles, reflecting the high importance and rarity of Class 3. The weight for Class 1 was set to 0.5 because Short but Sufficient Articles was the most over-represented category (so its loss is halved to prevent the model from over-focusing on it). The weighted cross-entropy loss thus became

$$L_{CE}^{*} = -\frac{1}{N}\sum_{i} w_{y_i} \log \hat{p}_{i,y_i}. \tag{4.3}$$

- **Incorporating an Ordinal Component: CE + MSE**

To address the **equal-penalty problem for ordinal errors**, the loss function was augmented with a Mean Squared Error (MSE) term on the class predictions. The idea was to introduce a penalty based on the distance between the predicted class and true class, treating the class labels as numerical values on an ordinal scale. In effect, this gives a harsher penalty to predictions that are far from the true label. Such a strategy aligns with approaches in other ordinal classification domains, where a class distance penalty is added to the loss to handle ordered labels. Formally, let the classes be encoded as ordinal values $\{0, 1, 2, 3\}$ for (Insufficient, Short, Standard, High-Quality). For a given sample i, define the predicted expected class (a single continuous value) as:

$$\hat{y}_i = \sum_{c=0}^{3} c \cdot \hat{p}_{i,c}, \tag{4.4}$$

which is the expectation of the class index under the model's predicted distribution $\hat{p}_{i,c}$. Here $c = 0$ corresponds to Insufficient Articles and $c = 3$ to High-Quality Articles. The ground-truth label y_i is likewise taken as an integer in $\{0, 1, 2, 3\}$. The mean squared error for this sample is $(\tilde{y}_i - y_i)^2$, and the MSE loss over the dataset is:

$$L_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2. \tag{4.5}$$

This term grows quadratically with the difference between predicted and true class indices. For instance, if an article of true class High-Quality Articles is predicted as class Insufficient Articles, the term $(\hat{y}_i - y)^2$ will be much larger than if it were predicted as Standard Articles. By summing an MSE term with the cross-entropy, the hybrid loss function encourages correct classification and additionally encourages predictions to be closer to the true class when misclassifications occur. The combined loss was formulated as:

$$L_{\text{hybrid}} = L_{\text{CE}}^{*} + \lambda L_{\text{MSE}}, \tag{4.6}$$

where λ controls the relative importance of the ordinal MSE term. This approach effectively blends classification and regression objectives. The expectation was that the cross-entropy

component would drive the model to predict the correct class, while the MSE component would make it more sensitive to the magnitude of errors, thereby penalizing wildly incorrect predictions disproportionately. Literature on ordinal classification supports this intuition: a loss that "penalizes misclassifications more severely when they diverge farther from the true class" can improve performance on ordered-label tasks.

Balancing the Loss Terms (Choosing λ)

A key step was determining an appropriate value for λ. If λ is too low, the MSE term would be negligible and the loss reduces to standard CE (failing to use the ordinal information), if λ is too high, the model might overly optimize the MSE (treating the task like regression) at the expense of classification accuracy. To calibrate λ, the average magnitudes of the two loss components were analyzed. After an initial training epoch, the expected values of each loss were computed:

$$\mathbb{E}[L_{\mathrm{CE}^*}] = \frac{1}{N}\sum_{i=1}^{N} L_{\mathrm{CE}}(i), \qquad \mathbb{E}[L_{\mathrm{MSE}}] = \frac{1}{N}\sum_{i=1}^{N} L_{\mathrm{MSE}}(i). \tag{4.7}$$

Their ratio

$$\alpha = \frac{\mathbb{E}[L_{\mathrm{CE}}]}{\mathbb{E}[L_{\mathrm{MSE}}]} \tag{4.8}$$

provides a rough scaling factor to make the two loss terms contribute equally on average. In our case, this ratio α was used as an initial guess for λ. (Intuitively, if cross-entropy loss values were, say, ten times larger than MSE values, one would set $\lambda \approx 10$ to balance them.) Starting from $\lambda \approx \alpha$, we then experimented with a range of λ around this value to fine-tune performance.

Experiments with MSE Weight

Several models were trained with different MSE weight values to evaluate their impact. Table 4.2 summarizes the weighted F1 results (the primary evaluation metric) for a range of λ values on the validation set:

MSE Weight λ	Weighted F1 (Validation)
0	0.69
0.5	0.70
1.0	0.721
2.0	0.705
3.0	0.693

Table 4.2: Performance for different MSE loss weightings for RetroMAE model.

As shown, the inclusion of the MSE term yielded a noticeable improvement over using pure cross-entropy ($\lambda = 0$). Performance peaked around $\lambda \approx 1.0$, which indeed was close to the calculated α in our case, confirming that balancing the two objectives was beneficial. A small MSE contribution (0.5) already improved the model's F1, and using equal weight ($\lambda = 1.0$) for CE and MSE gave the highest F1. Increasing λ beyond 1.0 did not help further; a very large MSE weight started to slightly degrade performance (likely because the model then over-emphasized predicting the exact ordinal value, at times sacrificing classification fidelity). The best model with $\lambda = 1.0$ demonstrated that the ordinal loss component successfully

reduced large misclassification errors - for example, the confusion matrix showed far fewer cases of confounding the lowest vs. highest quality classes once the MSE term was introduced, compared to the CE-only loss. Our final loss function combined this weighted CE with the MSE term:

$$L_{\text{final}} = \frac{1}{N}\sum_{i=1}^{N}(w_{y_i}\big(-\log \hat{p}_{i,y_i}\big) + 1.0 \times (\tilde{y}_i - y_i)^2). \tag{4.9}$$

- **Final Results and Advantages of the Hybrid Loss**

The CE + 1.0 × MSE with class weighting configuration was selected as the final loss function. This engineered loss function substantially outperformed the initial approaches. It effectively balanced the model's performance across all classes. Notably, the recall for the High-Quality Articles class improved markedly. The model was able to identify a considerable portion of the High-Quality Articles that were previously being missed. This can be attributed to the 20× class weight forcing the model to pay attention to class 3, in combination with the ordinal MSE term which dissuaded the model from conflating class 3 with much lower classes.

In other words, the final loss configuration addressed both major challenges: it distinguished large vs. small misclassification errors (due to the MSE term) and mitigated class imbalance (due to the class weights on the CE term).

In summary, through iterative loss function engineering, the thesis arrived at a hybrid loss that leverages the strength of cross-entropy for classification, infuses ordinal awareness via a regression term, and incorporates strategic weighting to handle class frequency skew. This nuanced loss function was pivotal in boosting the model's ability to mimic human-like judgments of article quality.

4.7 Tokenization and Sequence Handling

Each article in the dataset is first tokenized using the chosen tokenizer, converting the text into a sequence of subword tokens. The lengths of tokenized articles vary widely, so deciding on an appropriate maximum sequence length is crucial. To inform this decision, we analyzed the distribution of article lengths (in tokens) in the dataset. Figure 4.3 shows a histogram of token counts per article, which we use to guide our sequence handling strategy.

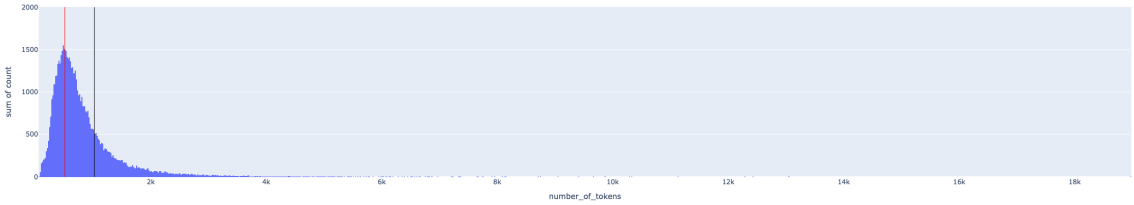


Figure 4.3: Distribution of token counts per article in the dataset. The red vertical line marks the 512-token cutoff and the black vertical line marks the 1024-token cutoff.

As illustrated in Figure 4.3, the token count distribution is strongly right-skewed. A significant portion of the articles have fewer than 512 tokens, so a limit of 512 tokens would allow us to process most articles without truncation. A substantial portion of articles have lengths between 512 and 1024 tokens, and beyond 1024 tokens there is a long tail of increasingly lengthy articles that are progressively rarer. In the histogram, the red vertical line marks the 512-token point and the black vertical line marks 1024 tokens, corresponding to the two sequence length cutoffs we considered.

Based on this distribution, we chose an initial maximum sequence length of 512 tokens and an extended sequence length of 1024 tokens. The 512-token limit covers the full content of most articles while keeping computational requirements manageable. Extending the limit to 1024 tokens allows the model to capture additional context from longer articles that exceed 512 tokens. Articles longer than 1024 tokens are relatively uncommon, so 1024 was deemed a reasonable upper bound. For any article exceeding the chosen maximum length, we apply truncation to fit the input size of the model, ensuring that we balance coverage of the article content with efficiency.

4.8 Training Methodology

After designing the hybrid loss function and addressing class imbalance, we proceeded to fine-tune the DeBERTaV3 model with a carefully planned training regime to mitigate rapid overfitting. Fine-tuning a large pre-trained model on a relatively small dataset can quickly lead to the model memorizing the training set. To counter this, our training methodology incorporated staged training, learning rate scheduling with warmup, and strong regularization through weight decay (L2 penalty) and dropout. All model training used the final loss function from Section 4.6 - the weighted cross-entropy combined with an MSE term (Equation 4.9) - to ensure the model optimized both classification accuracy and ordinal consistency during training.

Two-Stage Fine-Tuning

We adopted a staged training strategy to gradually introduce the full capacity of DeBERTaV3. In the first stage, we trained only the classifier head (the final linear layer and task-specific parameters) while keeping all pre-trained Transformer layers frozen. This allowed the new classification layer to learn basic separations between classes without immediately over-fitting the entire model to the limited data. We ran this head-only training for a short period until the head's performance plateaued. In the second stage, we unfroze the full DeBERTaV3 model and continued fine-tuning all layers. At this point, the classifier head was already reasonably calibrated, so the Transformer layers could adjust more subtly to improve performance. This two-stage approach meant the model started in a conservative regime and only later used its full complexity, reducing the risk of early overfitting. We found that the transition between these stages led to a noticeable improvement in validation loss - as soon as the Transformer layers were unfrozen and fine-tuning began, the model's validation loss dropped significantly (indicating it could fit the data better once allowed to update the base weights). We used the same optimizer and learning rate schedule across both stages, but reset the learning rate to a slightly lower value upon unfreezing to ensure stability (since large updates to all weights after unfreezing could destabilize training). By the end of stage two, the model was fine-tuned end-to-end, having benefited from a "warm start" in the classifier layer.

Learning Rate Warmup and Scheduling

A key element in preventing overfitting and training instability was the use of a tailored learning rate schedule. We employed a short warmup period at the start of training, during which the learning rate was gradually increased from a very low value to the target peak rate. In practice, we linearly ramped up the learning rate over the first 10% of training steps. This warmup phase avoids sudden large weight updates in the early iterations when the model's pre-trained weights are still adjusting to the new task. Without warmup, we observed that the validation loss could briefly spike early on - indicative of the model overfitting or diverging on easier patterns too quickly. The warmup helps "fight early overfitting" by starting training gently giving the optimizer time to find a stable direction before full step sizes are used. Once warmup was completed and the learning rate reached its maximum (we used a peak rate of 1e-05 for full-model training, typical for base-sized transformers), we switched to a **cosine decay scheduler** for the remainder of training. The cosine learning rate schedule gradually decreased the learning rate after the warmup, following a cosine curve down to a very low rate by the end of training. This scheduling ensured that as training progressed, updates became smaller and

more fine-grained, which helps the model converge to a minimum without overshooting. In effect, the high learning rate early on (after warmup) allows the model to make swift progress on fitting the data, while the continuously decreasing learning rate later allows convergence to stabilize and reduces the chance of the model overfitting in the final epochs. No learning rate restarts were used (i.e., we did not reset to high LR again), since our goal was a smooth decay to aid convergence. The combination of warmup and cosine annealing proved effective: the warmup prevented unstable jumps at start, and the slow tapering of learning rate toward zero acted as an implicit regularizer (making it harder for the model to overfit late in training, as it can only make tiny adjustments by then).

- **Regularization and Hyperparameters**

Along with the scheduling, we applied standard regularization techniques to further combat overfitting. We used the AdamW optimizer (Adam with decoupled weight decay) with a weight decay coefficient of 1e-3 on all Transformer weights. Weight decay acts as an L2 regularizer, continuously penalizing large weight magnitudes and thus discouraging the model from excessively complex or spiky weight patterns that could memorize the training data. This was especially important given the small dataset - without weight decay, the model could quickly drive some weights to extreme values to fit particular examples. We also retained dropout regularization in the model: DeBERTaV3's dropout rate of 15% was kept active during fine-tuning. Dropout adds noise to the training process by randomly zeroing some activations, which also helps prevent the model from relying too much on any one feature and thus improves generalization. Together, weight decay and dropout provided a strong regularizing effect throughout training. In addition, we weighted the loss function to emphasize the minority class, as described in Section 4.6. In each training batch, the loss from High-Quality Articles articles was multiplied by 20× (via the class weight in the cross-entropy term) to ensure those rare examples had a meaningful impact on the gradients. This class weighting, combined with the ordinal MSE component, was crucial for learning the minority class without overfitting the majority classes. We used a batch size of 16 articles and shuffled the training data on each epoch to present the model with a different mix of articles each cycle (reducing the chance of seeing examples in a fixed order, which can also lead to overfitting patterns). The training was run for a maximum of 5 epochs. We evaluated the model on the validation set at the end of each epoch (and additionally every 500 steps) and tracked the validation loss and F1-macro score. Indeed, we found that the model often began to overfit after only a few epochs - so we load the model from checkpoint that have the highest f1 score, which corresponded to roughly 18k-20k training steps in our setup.

The training dynamics under this methodology are illustrated in Figure 4.4, which plots the validation loss as a function of training steps. We can observe several key turning points in this curve that correspond to our training strategy. First, the initial phase (left part of the curve) shows the validation loss starting around 1.5 and then declining steadily as the classifier head is trained (while the base is frozen). This downward trend confirms that even with the base model frozen, the new head was learning useful decision boundaries. However, the decline in loss is gradual and modest (hovering around 1.4-1.5) during the head-only stage, and we see a bit of noise - this was expected, as a single linear layer can only capture so much without adjusting the underlying representations. The most prominent feature of the curve is the sharp drop in validation loss that occurs around the transition between stage one and stage two of training. Just before 10k steps on the plot, the validation loss plunges from the mid-1.4 range down to about 1.0. This coincides with the point at which we unfroze the

Figure 4.4: Validation loss over training steps.

Transformer layers and began fine-tuning the entire model. The sudden improvement (nearly 0.4 reduction in loss) confirms that once the full model was allowed to adapt, it very quickly fit the data distribution much better - a testament to DeBERTaV3's capacity to model complex patterns when fine-tuned. This sharp valley in the curve (reaching a minimum validation loss of about 0.95 around step 16,000) represents the optimal model during training. Notably, after this point, the curve begins to flatten and then rise, marking the onset of overfitting. As training continued beyond 20k steps, the validation loss started to increase again - slowly at first (between 20k-25k steps it oscillates upward around 1.2-1.3) and then more clearly climbing towards 1.35+ by 25k-30k steps. This upward turn indicates that the model began to memorize the training set at the expense of validation performance once it had exhausted the benefits of fine-tuning. In other words, after the initial gains, each additional training step was now slightly hurting generalization. We stopped training at this point and selected the best-performing checkpoint (the one at 16k steps where the validation loss was lowest and validation F1 highest) as our final model. It highlights the importance of our warmup and staged approach - the curve's smooth descent and single clear minimum suggest the training was well-behaved (no oscillations or chaotic jumps), and the worst overfitting was kept at bay until late in training. Had we not used these techniques, the validation loss might have bottomed out much earlier and risen more sharply.

In summary, our training methodology balanced aggressive optimization (to quickly fit the data) with regularization and cautious scheduling (to avoid overfitting). By first training the classifier head and then fine-tuning the full model with a warmed-up, decaying learning rate, we achieved a stable training process that fully utilized the hybrid loss's benefits. The use of weight decay and dropout further kept the model's complexity in check, while class weighting ensured the model learned from minority class examples rather than ignoring them. This careful training setup was crucial for the model to achieve high validation performance. Ultimately, the best model - trained with the hybrid CE+MSE loss and these methodological safeguards - attained a significantly improved generalization compared to baseline attempts. It maintained a low validation loss until the point of overfitting, demonstrating that the model learned a robust representation of journalistic quality features without succumbing too early to noise in the training data.

4.9 Model Evaluation

After adding several features from Section 4.3, the DeBERTaV3 model achieved a validation weighted F1 score of 0.766. The class-wise F1 scores were as follows:

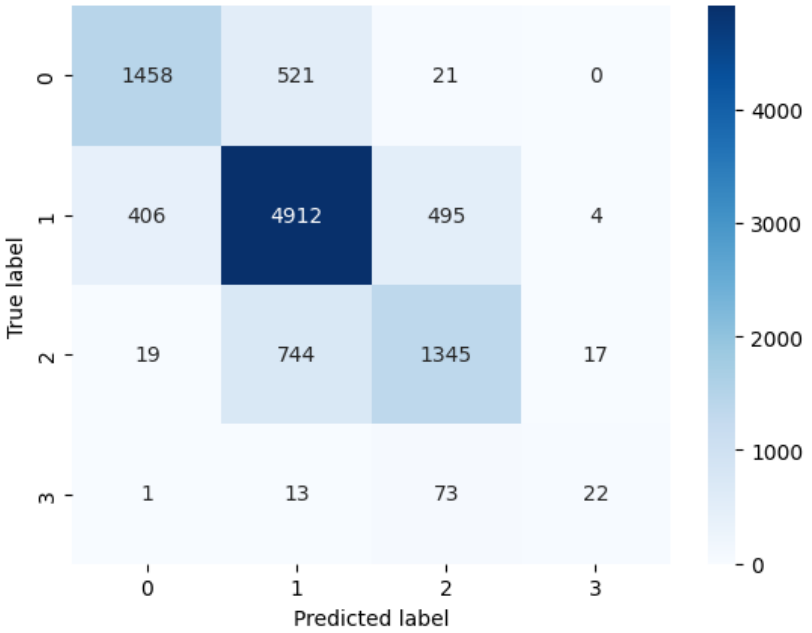


Figure 4.5: Confusion matrix of the final model's predictions on the validation set, highlighting the remaining misclassifications between classes.

- Class 0: F1 = 0.755
- Class 1: F1 = 0.82
- Class 2: F1 = 0.660
- Class 3: F1 = 0.20
- Accuracy: 0.77
- Weighted F1: 0.766

The confusion matrix in Figure 4.5 shows that the model still often confuses Class 3 examples with Class 1 and Class 2. However, the overall balance of predictions has improved compared to earlier results, indicating that the hand-crafted features had a positive effect on performance.

4.10 CatBoost

We employed Categorical Boosting (CatBoost) inrtodused by Prokhorenkova et al. 2018, a gradient boosting classifier, to predict article quality using the engineered feature set. CatBoost was chosen for its ability to handle categorical data natively and its robustness on imbalanced datasets. In our case, this meant we could include metadata and content-based attributes together without extensive preprocessing. For example, CatBoost can directly ingest a categorical feature like channel ID (the article's source channel) by using an ordered encoding scheme that computes target statistics from prior data points, thereby avoiding target leakage.

Another architectural advantage of CatBoost is its use of symmetric (oblivious) decision trees, where all splits at a given tree level use the same condition. This yields consistent, fast models and, coupled with the ordered boosting algorithm, helps reduce overfitting. These properties made CatBoost well-suited to our feature-based approach.

Motivation and Suitability

The motivation for using CatBoost was twofold.

First, it seamlessly handles the mix of linguistic complexity metrics, structural composition indicators, readability scores, and metadata features from Section 4.3. Unlike some algorithms, it does not require one-hot encoding or manual scaling of categorical inputs, preserving useful information encoded in features like genre or source.

Second, CatBoost's robustness to class imbalance was critical: our dataset's quality classes were highly skewed. CatBoost provides built-in options for balancing classes (e.g. an auto_class_weights="SqrtBalanced" setting) to ensure minority classes are not ignored. This capability, combined with the algorithm's general resilience to overfitting, made it a strong candidate for our task.

Hyperparameter Tuning

We conducted a thorough search over CatBoost's hyperparameters to achieve optimal multi-class performance. This included comparing different loss functions (evaluating CatBoost's standard multi-class loss vs. one-vs-all objective), adjusting the number of boosting iterations, tuning tree depth, L2 regularization (via the l2_leaf_reg parameter), and modifying random_strength to manage overfitting.

After evaluating various configurations, we found that the best results were achieved using CatBoost's built-in MultiClass loss function with 1500 boosting iterations. Tree depth was kept moderate, generally in the range of 6 to 8, to strike a balance between model complexity and generalization. A small L2 regularization value helped prevent the model from fitting overly specific leaf values. Additionally, a non-zero random_strength value was selected to introduce beneficial randomness into the tree-splitting process, which smoothed the decision boundaries.

We also enabled early stopping with a patience of 200 rounds, halting training if the model's validation performance stopped improving for 200 consecutive iterations. This not only helped avoid overfitting but also supported our choice of 1500 trees, as the validation score typically plateaued around that point. All hyperparameters were jointly fine-tuned to maximize the weighted F1 score on the validation set, ensuring balanced performance across all classes.

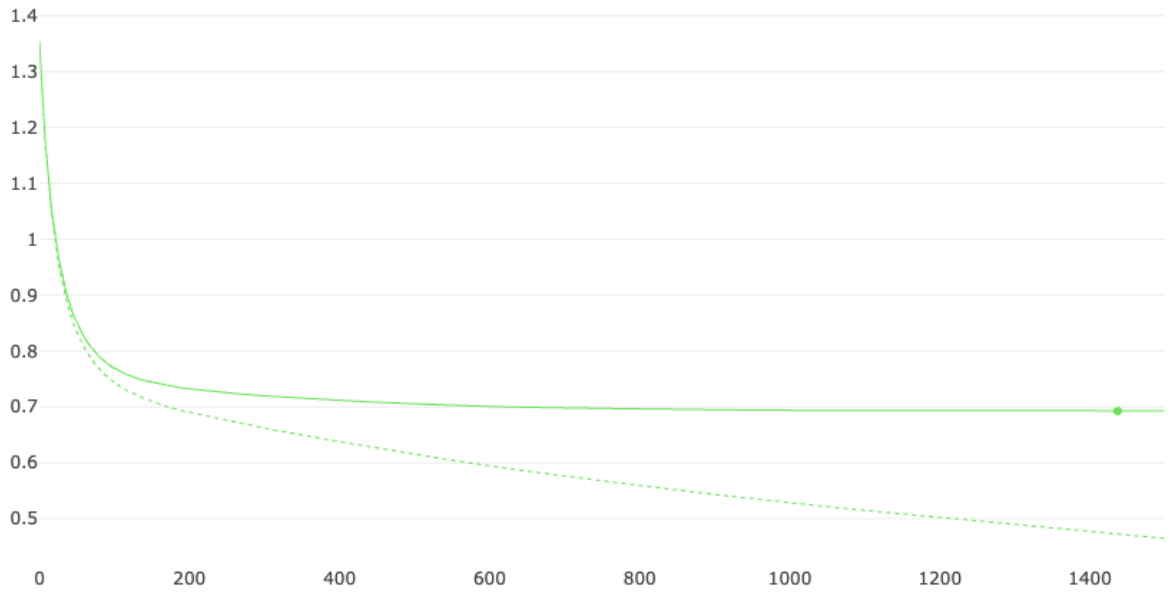


Figure 4.6: CatBoost training and validation loss over iterations.

Figure 4.6 plots the training and validation log-loss. The curves flatten well before the stopping point, justifying the chosen iteration count.

- **Architecture and Feature Processing**

We configured CatBoost to utilize all four feature groups described earlier. Internally, the model builds symmetric decision trees that split on the most informative features at each depth. The ordered boosting procedure meant that categorical features were transformed on the fly using only preceding training examples, which prevented the model from learning spurious correlations. One categorical feature, channel ID, was initially included among the inputs and, as expected, CatBoost identified it as extremely predictive of quality (in fact, in preliminary runs channel ID alone accounted for roughly 70% of total feature importance). This indicated that the source/publisher was a dominant signal for quality in our data. However, relying on this metadata goes against the goal of assessing content intrinsic quality. We therefore removed channel ID in the final model. After this adjustment, CatBoost focused on the content-based features (linguistic, structural, readability, etc.), and no single feature dominated the importance. The model could thus base its predictions on a combination of stylistic and structural characteristics of the text rather than a possibly biasing identifier.

- **Feature-Group Importance**

To understand the contribution of each feature group to the final model, we computed the mean feature importance across all features in each group. This was done by aggregating the importance scores assigned by CatBoost to each feature and normalizing them to sum to 1. The results are shown in Table 4.3, which illustrates the relative importance of each feature group in the final model.

The table shows that none of the groups dominated the model's decisions, indicating that CatBoost was able to learn from a diverse set of features. The structural composition and

Group	Share of Total Importance
Structural composition	$\approx 29\,\%$
Readability metrics	$\approx 26\,\%$
Linguistic complexity	$\approx 21\,\%$
Metadata & tags	$\approx 24\,\%$

Table 4.3: Feature Group Importance Breakdown

readability metrics were the most important, suggesting that the model relied heavily on these aspects to assess article quality. Linguistic complexity and metadata also contributed significantly, but to a lesser extent. This balanced feature importance distribution aligns with our expectations: high-quality articles should exhibit a combination of good structure, readability, and linguistic sophistication.

Performance and Practical Considerations

With weighted F1=0.725 the final CatBoost model lags the neural network yet surpasses simpler baselines and remains attractive for two reasons:

1. Speed – CPU inference takes < 3 ms per article, enabling real-time scoring.

2. Interpretability – per-instance SHAP values help editors understand which textual properties influenced a given prediction.

Consequently, CatBoost serves as a complementary component in the ensemble pipeline, offering a lightweight, transparent alternative to transformer-based models.

4.11 Hybrid Model: CatBoost with Language Model Features

Based on the promising results from the standalone CatBoost model, we next experimented with a hybrid approach that combines CatBoost with outputs from BERT-like models. The motivation was to leverage the model's understanding of text to improve classification performance, particularly to better recognize the minority High-Quality Articles. Initial CatBoost experiments showed potential but struggled with High-Quality Articles due to its rarity, so we explored whether integrating features derived from an Pre-trained Language Models could boost the model's sensitivity to these high-quality articles. Table 4.4 summarizes the performance of the hybrid model on the validation set, comparing it to the standalone CatBoost and DeBERTaV3 models.

In the first integration attempt, we incorporated learned DeBERTaV3 embeddings as additional features for the CatBoost classifier. In this setup, each article's text was encoded into a dense vector representation by the DeBERTaV3, and this vector was appended to the existing feature set before training CatBoost. The intuition was that the embedding might capture complex semantic attributes of the text beyond our manually engineered features. However, this approach did not yield any improvement – the CatBoost model's validation metrics remained essentially unchanged when the embedding features were added. The lack of uplift suggested that either the embedding information was redundant with our current features or CatBoost could not effectively leverage the high-dimensional embedding in this configuration.

Next, we tried a different strategy: using the DeBERTaV3's predicted class label as an input feature. In this second experiment, the DeBERTaV3 was used as a separate classifier

to predict a quality class for each article (e.g. by analyzing the text and outputting a class 0–3 label). We then fed that predicted label (as a categorical feature) into CatBoost along with all other features. The rationale was that the DeBERTaV3's prediction, even if not perfect, might cue CatBoost to certain linguistic patterns or content signals that align with quality. This method produced a slight improvement in overall performance – the weighted F1 score on the validation set rose to about 0.76. This was a modest gain over the baseline and indicated that the DeBERTaV3's judgment provided some useful signal. However, the improvement was not uniform across classes: High-Quality Articles in particular remained difficult to detect, with its precision and recall still very low. In other words, giving CatBoost the single class prediction from the DeBERTaV3 wasn't enough to substantially help with the rare high-quality articles.

Finally, our third and most successful approach was to supply CatBoost with the full DeBERTaV3 output logits for each class, rather than just the single best label. In this setup, the DeBERTaV3 produces a probability or confidence score for each class (0, 1, 2, 3) for a given article – essentially a vector of raw prediction scores across all classes, often referred to as logits. We included these four logit values as additional features alongside the existing engineered features and then trained the CatBoost classifier on this enriched feature set. This hybrid model formulation yielded the best results: the CatBoost + DeBERTaV3-logits model achieved a weighted F1 score of approximately 0.78 on the validation data, the highest of all tested configurations. The improvement suggests that providing the full distribution of the DeBERTaV3's predictions gave CatBoost more nuanced information. Instead of a single guess, CatBoost could learn from the DeBERTaV3's confidence levels in each class, leveraging cases where the DeBERTaV3 "hedged" between Standard Articles and High-Quality Articles, for example. This allowed the hybrid model to better differentiate articles, especially in borderline cases, and boosted overall classification performance.

Model	**Weighted F1**	**Accuracy**
CatBoost	0.725	0.734
DeBERTaV3	0.766	0.77
CatBoost + DeBERTaV3 embedding	0.74	0.731
CatBoost + DeBERTaV3 class	0.76	0.78
CatBoost + DeBERTaV3 logits	**0.779**	**0.78**

Table 4.4: Performance of the hybrid model on the validation set.

The matrix from Figure 4.7 shows that Class 1 (the majority class) had the highest number of correct predictions, with most of its errors being confusion with Class 2. In fact, the two most frequent misclassifications occurred between Class 1 and Class 2, reflecting the challenge in distinguishing adjacent quality levels. Class 3 remained the hardest category to identify correctly – the model only achieved 28 true positives for Class 3, and the majority of articles that were actually Class 3 were misclassified as Class 2. Class 0 and Class 2 also saw some confusion between each other and with Class 1, though to a lesser extent than the Class1 – Class2 overlap. This breakdown confirms that while performance for the frequent classes (0, 1, 2) was moderate to high, the High-Quality Articles category still suffered from low precision and recall due to its scarcity and the model's difficulty in separating it from Class 2.

In summary, our experiments with integrating DeBERTaV3-derived features demonstrated that embeddings or single-label predictions alone did not significantly enhance the

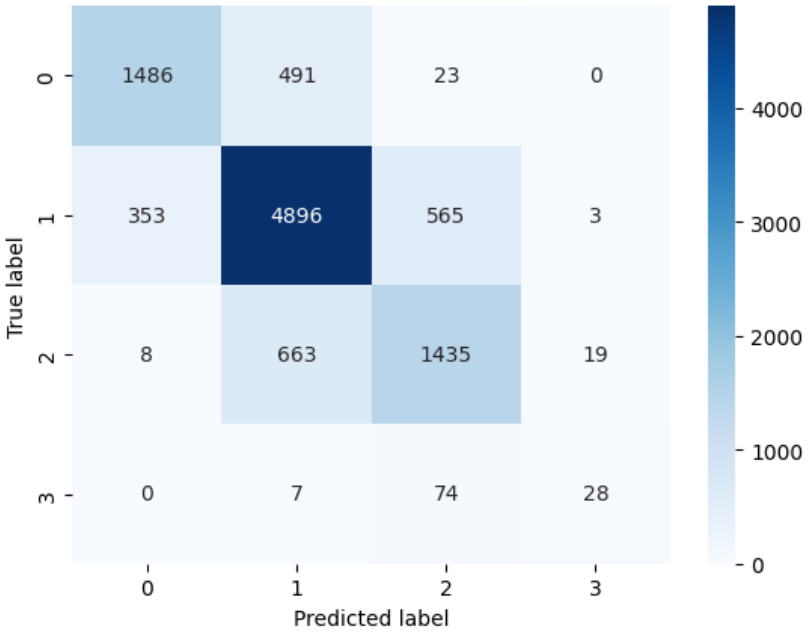


Figure 4.7: Confusion matrix of the hybrid model's predictions on the validation set.

CatBoost model. In contrast, providing the DeBERTaV3 full logits as features led to a notable improvement in classification performance. This hybrid approach successfully combined the DeBERTaV3's language understanding with CatBoost's structured learning, yielding the highest overall F1 score. Therefore, the CatBoost model enriched with DeBERTaV3 logits was selected as the final model configuration for the task, as it offered the best balance of improved accuracy (especially overall F1 ≈ 0.78) and the ability to handle all classes, including the challenging Class 3, better than previous attempts.

5 Experiments and Results

5.1 Experimental Setup

We fine-tuned each of the four pre-trained language models (BROWCzech, RetroMAE, DeBERTaV3 base, and DeBERTaV3 large) on our journalistic dataset. All models used the same data splits (56,788 articles for training, with separate validation and test sets as described in Chapter 2). Training was performed in two stages: first we trained only the classification head for 6 epochs, then we unfroze the full model and trained all layers for 5 more epochs. This two-stage schedule (11 total epochs) was chosen to stabilize training. The models were implemented in **PyTorch**(Paszke 2019) and fine-tuned using the **Hugging Face Transformers** library (Wolf et al. 2020). We used mixed-precision (FP16) on an NVIDIA H100 PCIe GPU; the total training time for all experiments was about 27 hours. To support experimentation, we logged metrics with **MLflow** and used **Pandas** tools to handle the data; class labels were encoded with **torchnlp.encoders.LabelEncoder**.

- **Dataset**: Training set of 56,788 articles; validation and test splits as in Chapter 2.
- **Environment:** PyTorch and Hugging Face Transformers, mixed-precision training (FP16) on an NVIDIA H100 GPU (27 GPU-hours).
- **Training procedure:** Two-stage fine-tuning – 6 epochs training of the new classification head (base layers frozen), followed by 5 epochs of full-model training.
- **Tools:** MLflow (experiment tracking), pandas/csv (data handling), torchnlp LabelEncoder (label encoding).

5.2 Neural Model Results

The test performance of the four neural models is compared in Table 5.1. The weighted F1 score of each model are shown. As expected, BROWCzech (a simpler encoder) had the lowest performance, while the DeBERTaV3 large models achieved higher accuracy. DeBERTaV3 base outperformed RetroMAE and BROWCzech, and DeBERTaV3 large improved slightly further. The best neural model was DeBERTaV3 large, with a weighted F1 around 0.766 and accuracy around 0.77.

This high-level results are consistent with our expectations. DeBERTaV3's stronger architecture led to better classification of article quality. In particular, the large model's extra capacity helped capture nuanced journalistic features. Still, all models showed difficulty with the rare High-Quality Articles. To illustrate the errors, Figure 5.1 shows the confusion matrix for the DeBERTaV3 large model on the test data. We see that the most common class (Short but Sufficient Articles) had the most true positives, but it was sometimes confused with Standard Articles (Class 2). High-Quality Articles (Class 3) remains very hard: only a small number of high-quality articles were correctly identified, and many were misclassified as Class 1 or 2. This reflects the severe class imbalance (high-quality articles are ¡1% of the data).

5.3 CatBoost Model

We also trained a CatBoost classifier using features created in Section 4.3. The CatBoost model required only about 1 minute of training on the same GPU. This model achieved a

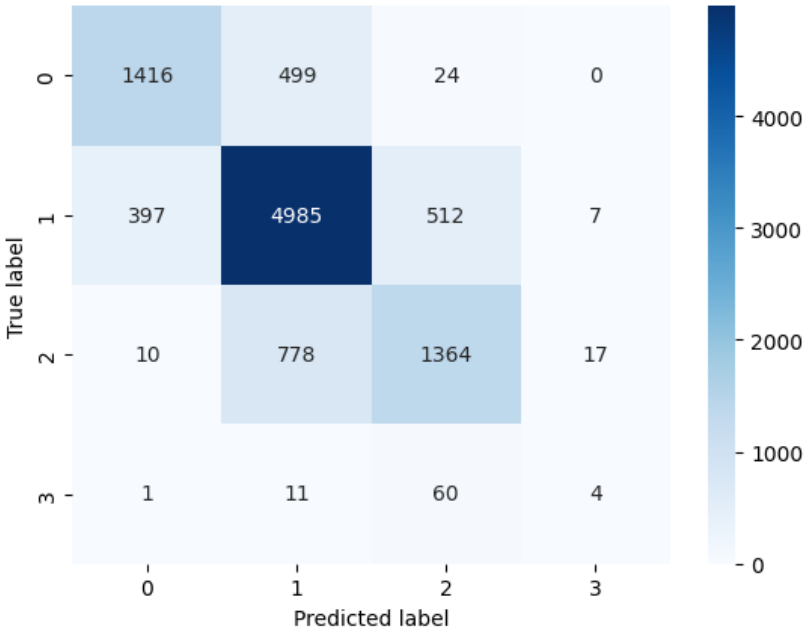


Figure 5.1: Confusion matrix for the DeBERTaV3 large model on the test set.

weighted F1 around 0.716 (accuracy 72,8%). The CatBoost model was fast to train and already competitive with the neural models. However, it did not outperform the best neural model (DeBERTaV3 large). The CatBoost model's confusion matrix (Figure 5.2) shows that it struggled with the same classes as the neural models, particularly High-Quality Articles.

5.4 Hybrid Model Results

The final hybrid model combined the strengths of both approaches: we fed the four logit scores from DeBERTaV3 large into the CatBoost classifier alongside other features. This hybrid (ensemble) model gave the best results of all experiments. On the test set, the hybrid model achieved a weighted F1 score of about 0.779 and an accuracy of roughly 71,1%, outperforming each individual model. These values surpass the single-model baselines and approach the human agreement upper bound (≈80%) noted in Chapter 4.1. The results demonstrate that CatBoost could effectively use the extra information in the DeBERTaV3 outputs to correct some mistakes. Figure 5.3 shows the confusion matrix for the hybrid model on the test data. The matrix indicates improved true-positive counts in most classes. Misclassifications between Class 1 and 2 remain the largest source of error, but overall the model is more calibrated. The hybrid model's gains are especially clear in classifying borderline articles: by seeing the full DeBERTa prediction vector rather than just a single guess, CatBoost learned to distinguish subtle cases better. This is evident in the confusion matrix, where fewer Standard Articles are mistaken for Short but Sufficient Articles compared to the purely neural model.

5.5 Summary of Results

In summary, the performance ranking of the models shown in the Table 5.1 were as follows: Hybrid (DeBERTaV3 large + CatBoost) > DeBERTaV3 large > DeBERTaV3 base > RetroMAE > BROWCzech > CatBoost. The hybrid model achieved the best weighted

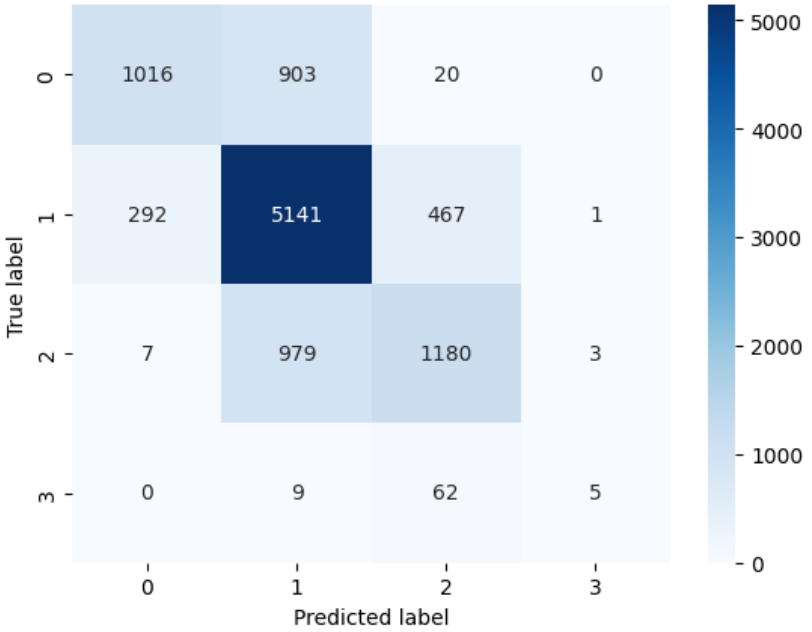


Figure 5.2: Confusion matrix for the CatBoost model on the test set.

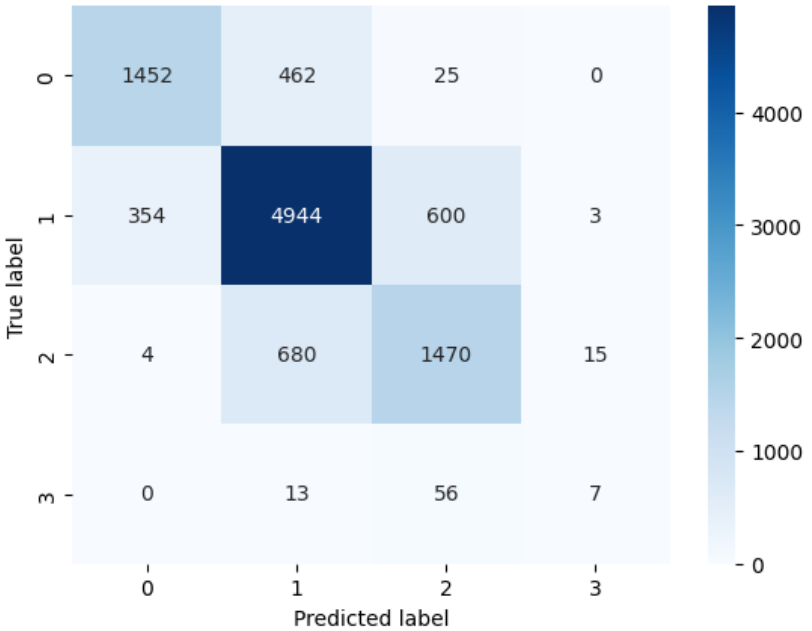


Figure 5.3: Confusion matrix for the hybrid model (DeBERTaV3 large + CatBoost) on the test set.

F1 (≈0.779) and accuracy (≈0.781), thanks to combining neural embeddings with gradient boosting. The CatBoost classifier alone was fast to train (≈1 minute) and already competitive, but the ensemble yielded a clear improvement. Across all models, we consistently measured weighted F1 and accuracy on the test set. The improvement from the best single model (DeBERTaV3 large) to the hybrid model is small but consistent, indicating that the additional decision-tree model helps correct a subset of errors. These results, together with the confusion matrices and charts above, show that our automated system achieves near human performance on Czech news quality classification (approaching the human agreement benchmark).

Model	**Weighted F1**	**Accuracy**
BROWCzech	0.71	0.70
RetroMAE	0.72	0.72
DeBERTaV3 base	0.724	0.73
DeBERTaV3 large	0.74	0.738
DeBERTaV3 large (features)	0.766	0.77
CatBoost (features)	0.716	0.728
Hybrid (DeBERTaV3 large + CatBoost)	**0.779**	**0.781**

Table 5.1: Test performance of the models on the validation set

6 Discussion and Future Work

6.1 Discussion

This thesis set out to build an automated system for classifying the quality of news articles. The final chosen model performed well overall, indicating that the approach is sound. In most cases, the system correctly distinguished between the predefined quality levels (Insufficient, Short but Sufficient, Standard, High-Quality). The model's predictions closely matched the categories that human editors assigned, showing that it learned to apply the editorial criteria consistently. In particular, the combination of a transformer-based language model (DeBERTaV3) and carefully engineered features proved effective in capturing important aspects of article quality, such as readability, structure, and presence of references.

After training the model, a detailed error analysis was carried out to understand its remaining mistakes. This analysis revealed that not all errors were due to model limitations. In fact, some articles in the dataset were found to have incorrect or inconsistent labels. These label errors were caused by unclear annotation guidelines: different annotators sometimes disagreed on the proper quality category for an article. For example, an article with several references and clear structure might have been labeled as a lower-quality class because the instructions were ambiguous. Such inconsistencies in the data made it harder for the model to learn the true distinctions.

To address this issue, the plan is to improve the annotation process. Clearer guidelines will be developed so that annotators can label articles more consistently. This may involve refining the definitions of each quality class and providing more examples or training for annotators. The dataset can then be reviewed and corrected where needed. Improving the label quality is expected to help the model's performance, because it will train on more reliable data and the evaluation will be more meaningful.

Despite the data challenges, the model showed several strengths. It handled the class imbalance in the data through techniques (as described in earlier chapters) so that even the rare High-Quality class was recognized reasonably well. The engineered features based on journalistic standards (such as measures of textual complexity, citation use, and article structure) appeared to contribute positively, as the model could detect the intended quality signals. Some confusion still remained between adjacent classes (for instance, telling apart a very good article from an excellent one), but this is also a difficulty for human editors. Overall, the results indicate that automated classification can largely mimic human judgments, although some care is needed. The model should be seen as a tool to support editors, providing consistent and quick assessments, rather than a replacement for human review.

In summary, the discussion highlights that the system achieves a high level of accuracy in classifying article quality, roughly on par with human consistency. The main limitation uncovered was the noise in the training labels, which will be improved. The insights gained (especially about data quality and relevant features) point to clear steps for making the system even better.

6.2 Future Work

Several directions can be pursued to enhance and extend this research:

- Improve annotation quality. Refine the labeling guidelines and re-examine the dataset to correct inconsistent labels. Clearer instructions for annotators and a review process (for example, multiple editors checking each article) will reduce noise in the data. Higher-quality annotations will help the model learn more accurate distinctions.
- Integrate into the editorial workflow. Deploy the model in the Seznam.cz production environment so that it provides real-time quality ratings for new articles. This could involve building an automated pipeline or dashboard where editors see the model's predictions. In practice, the system's output could guide editors to focus on borderline cases or flag problems early. Gathering feedback from actual editors using the tool will help improve the system.
- Enhance explainability. Incorporate techniques to make the model's decisions more transparent. For example, use feature-attribution methods (such as SHAP or LIME) to highlight which parts of an article influenced its quality score. Such explainability tools would allow users to understand why the model made a certain prediction, increasing trust and enabling further refinement of features.
- Expand data and features. Collect more data to cover a broader range of content, possibly including articles from different time periods or additional sources. Additional features could be explored, such as social engagement metrics or multi-modal signals if images are present. More data and richer inputs can improve model robustness.
- User studies and iterative refinement. Conduct user studies with human editors interacting with the system to identify usability issues and desired improvements. Based on this feedback, refine the model, features, and interface. This iterative development will ensure the tool meets practical editorial needs.

By pursuing these future tasks, the automated quality classification system can become more accurate, reliable, and useful. Improved data quality, seamless integration, and better interpretability will all help bridge the gap between the model and real-world editorial use, making it a valuable assistant for maintaining high journalistic standards.

7 Conclusion

Summarize the achieved results. Can be similar as an abstract or an introduction, however, it should be written in past tense.

8 References

Calvo-Rubio, Luis M. and José L. Rojas-Torrijos (2024). "Criteria for journalistic quality in the use of artificial intelligence". In: *Communication & Society* 37.2, pp. 247–259. DOI: 10.15581/003.37.2.247-259. URL: https://www.researchgate.net/publication/380169433_Criteria_for_journalistic_quality_in_the_use_of_artificial_intelligence.

Quantum, Sophia (2024). *Automated content quality assessment with AI*. https://medium.com/@SophiaQuantum/automated-content-quality-assessment-with-ai-6719b326ab15. Accessed: January 2025.

Xiao, Shitao et al. (2022). "RetroMAE: Pre-training retrieval-oriented language models via masked auto-encoder". In: *arXiv preprint arXiv:2205.12035*.

González-Gallardo, Carlos et al. (2021). "Stance detection with BERT embeddings for credibility analysis". In: *Procesamiento del Lenguaje Natural* 67, pp. 169–180. URL: https://asu.elsevierpure.com/en/publications/stance-detection-with-bert-embeddings-for-credibility-analysis-of.

He, Pengcheng, Jianfeng Gao, and Weizhu Chen (2021). "Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing". In: *arXiv preprint arXiv:2111.09543*.

Clark, Kevin et al. (2020). "Electra: Pre-training text encoders as discriminators rather than generators". In: *arXiv preprint arXiv:2003.10555*.

He, Pengcheng et al. (2020). "Deberta: Decoding-enhanced bert with disentangled attention". In: *arXiv preprint arXiv:2006.03654*.

Minaee, Shervin et al. (2020). "Deep Learning Based Text Classification: A Comprehensive Review". In: *arXiv preprint arXiv:2004.03705*. URL: https://arxiv.org/abs/2004.03705.

Wolf, Thomas et al. (2020). "Transformers: State-of-the-art natural language processing". In: *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, pp. 38–45.

Devlin, Jacob et al. (2019). "Bert: Pre-training of deep bidirectional transformers for language understanding". In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186.

Liu, Yinhan et al. (2019). "Roberta: A robustly optimized bert pretraining approach". In: *arXiv preprint arXiv:1907.11692*.

Paszke, A (2019). "Pytorch: An imperative style, high-performance deep learning library". In: *arXiv preprint arXiv:1912.01703*.

Schmidt, Miriam and Eva Zangerle (2019). "Article Quality Classification on Wikipedia: Introducing Document Embeddings and Content Features". In: *Proceedings of the 15th International Symposium on Open Collaboration, OpenSym '19*. New York, NY, USA: Association for Computing Machinery. ISBN: 9781450363198. URL: https://dl.acm.org/doi/10.1145/3306446.3340831.

Potthast, Martin et al. (2018). "A stylometric inquiry into hyperpartisan and fake news". In: *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, pp. 231–238. URL: https://ojs.aaai.org/index.php/AAAI/article/view/5386/5242.

Prokhorenkova, Liudmila et al. (2018). "CatBoost: unbiased boosting with categorical features". In: *Advances in neural information processing systems* 31.

Lin, Tsung-Yi et al. (2017). "Focal loss for dense object detection". In: *Proceedings of the IEEE international conference on computer vision*, pp. 2980–2988.

Vaswani, Ashish et al. (2017). "Attention is all you need". In: *Advances in neural information processing systems* 30.

Hendrycks, Dan and Kevin Gimpel (2016). "Gaussian error linear units (gelus)". In: *arXiv preprint arXiv:1606.08415*.

Lacy, Stephen and Tom Rosenstiel (2015). *Defining and Measuring Quality Journalism*. https://www.jwac.org/Lacy.pdf. Accessed: January 2025.

Pitler, Emily and Ani Nenkova (2008). "Revisiting readability: A unified framework for predicting text quality". In: *Proceedings of the 2008 Conference on Empirical Methods in Natural Language Processing*, pp. 186–195. URL: https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=a80837b2bbe847aa9ce82ea9a14d23d82b4e4119.

A Appendix A